Task Allocation via Verifiable Market Games

Riyaz Ahuja

March 26, 2026

Mellon College of Science
Department of Mathematics
Carnegie Mellon University
Pittsburgh, Pennsylvania 15213

Thesis Committee:

Dr. Prasad Tetali, Chair
Dr. Jeremy Avigad
Dr. Sean Welleck

Thesis submitted in partial fulfillment of the requirements for the degree of Master of Science in Mathematics

© Riyaz Ahuja, 2026

Dedicated to todo

Acknowledgments

todo

Abstract

Mathematical discovery is not a sequence of isolated proofs, but a distributed process across mathematicians and their expertise. We therefore model mathematical development as a scheduling and resource allocation problem, formalizing it as the Interdependent Task Allocation Problem (ITAP). Within a centralized-control regime, we study ITAP's complexity and approximation-theoretic structure. However, when agent capabilities are private, this regime becomes fragile, suggesting a decentralized formulation in which coordination is strategic rather than purely combinatorial is more tractible. We formalize this as a Verifiable Allocation Market POSG (VAMP) and show that exact equilibria computation is intractible in general, motivating a multi-agent reinforcement learning framework that approximates no-regret behavior in the empirical meta-game. We evaluate this model on synthetic ITAP environments against centralized baselines and apply the framework for theorem proving in Lean, showing that it improves multi-agent strategy amongst LLM baselines on research-level theorem proving tasks.

Table of Contents

List of Figures vii

List of Tables viii

1 **Introduction** 1

1.1 Motivation 1

1.2 Core Contributions 3

2 **Scheduling Theory and Task Allocation** 5

2.1 Classical Scheduling Theory 5

2.2 Examples 14

2.3 Extensions and Generalizations 17

3 **The Interdependent Task Allocation Problem** 21

3.1 Centralized ITAP 21

3.2 Approximation Theory 29

3.3 Complexity Theory 29

3.4 Implications and Limitations 29

4 **VAMP — Formal Definition and Properties** 30

4.1 Mechanistic Impossibility and Decentralization of ITAP 30

4.2 Partially Observable Stochastic Games 30

4.3 VAMP Preliminaries 33

4.4 Verifiable Allocation Market POSG 41

4.5 Decentralized ITAP-VAMP Correspondence 48

4.6 Equilibria and Tractibility 48

5 **Multiagent Reinforcement Learning for VAMPs** 49

6 **Implementation** 50

6.1 Assumptions 50

6.2 Proof Kernel ($\mathcal{K}_{\text{proof}}$). 53

6.3 Conjecture Kernel ($\mathcal{K}_{\text{conj}}$). 55

6.4 Query Model. 58

6.5 Market Mechanisms 62

7 **Experiments** 67

8 **Application to Neural Theorem Proving** 68

9 **Conclusion** 69

Bibliography 70

List of Figures

Figure 1.1 Schematic dependency graph for the Lean formalization effort around Fermat's Last Theorem. The intent is to visualize a theorem not as an isolated statement, but as the endpoint of a large directed dependency structure spanning multiple mathematical domains. Such a graph highlights why theorem proving, formal verification, and related activities may be viewed as allocation problems over interdependent tasks. 2

List of Tables

Chapter 1

Introduction

1.1 Motivation

In 1637, Pierre de Fermat famously conjectured that for $n > 2$, there are no positive integer solutions to the equation $x^n + y^n = z^n$. Despite Fermat's claim to have a proof of this statement, his textbook margins provided too little space for his supposed proof, and thus this conjecture remained unproven for 358 years. It was only in 1994 that Andrew Wiles, building on decades of work by many mathematicians across domains, finally proved Fermat's Last Theorem.

Wiles' proof of Fermat's Last Theorem did not arise from a direct attack on the original equation. Instead, it emerged from deep developments and decades of study of domains seemingly - at first glance - far removed from the scope of the original conjecture. Throughout the 20th century, the theory of elliptic curves and modular forms was developed extensively, not only for the purposes of resolving FLT, but rather for a more fundamental understanding of the connections between arithmetic and geometric objects (elliptic curves over $\mathbb{Q}$) with complex analysis. Indeed, even Wiles' own proof of FLT was but a corrolary of his proof of the deeper Taniyama-Shimura conjecture, which was developed as a central question of the unification of large branches of mathematics. And yet, without these foundational contributions to the theory – motivated not by the resolution of a particular conjecture but rather in search of a deeper understanding of the mathematical landscape – the proof of FLT would not have been conceivable, as the language to discuss it would not exist.

TODO POLISH

[Now discuss how this pattern is common in mathematics, and how it is the fundamental collaborative, community efforts that lead to these large breakthroughs. Looking at the dependency graph for the Lean formalization of FLT, we see that the proof of a single target theorem may require the coordinated development of many supporting results across multiple domains that did not exist at the time of conjecture [fig 1]. And to a certain extent, this

Placeholder for dependency graph of the Lean FLT project.

Figure 1.1. Schematic dependency graph for the Lean formalization effort around Fermat's Last Theorem. The intent is to visualize a theorem not as an isolated statement, but as the endpoint of a large directed dependency structure spanning multiple mathematical domains. Such a graph highlights why theorem proving, formal verification, and related activities may be viewed as allocation problems over interdependent tasks.

pattern of corpus creation directly leading to major breakthroughs is common even beyond mathematics, as these corpora provide the infrastructure and language that shape the way we think about problems and their solutions. [Like the meme about god being a shepherd, pilot, author, or computer scientist]]

[Now, with the acceration of computational tooling and the rise of machine learning, we are seeing the rapid emergence of new tools for trustworthy automation and scaled collaboration in mathematics, and the integration of these tools through agentic AI systems. These systems are increasingly being used to tackle large, interdependent bodies of mathematical work by high profile scientists, and are being deployed at scale for the formalization of research-level mathematics. However, these systems are still in their infancy, viewing mathematical development as a search problem rather than an allocation problem.]

[Towards the advancement of such digitized mathemtical ecosystems, we interpret the problem of mathematical development as an instance of a more general problem of allocating heterogeneous, distributed agents over a large pool of interdependent tasks: proving, conjecturing, collaborating, or even competing. Namely, this thesis states and studies this interdependent task allocation problem, both in centralized and decentralized regimes, and develops a game-theoretic framework for the latter in which coordination is mediated by a market mechanism. We study equilibria in this game, and develop learning-based methods for approximating weak equilibria behavior in large instances. We implement and evaluate these methods on synthetic environments, and we instantiate the framework in neural theorem proving within Lean, studying its performance on research-level theorem proving tasks against agentic LLM baselines.]

1.2 Core Contributions

The main contributions of this thesis are as follows.

1. We formalize the *Interdependent Task Allocation Problem* (ITAP), a framework for allocating heterogeneous agents over tasks linked by logical or operational dependencies, with support for recursive task generation.

2. We distinguish two coordination regimes for ITAP, centralized and decentralized, and identify the structural assumptions that separate them.

3. In the centralized regime, we relate ITAP to scheduling and assignment problems with dependency constraints, and derive complexity-theoretic and approximation-theoretic consequences from this connection.

4. In the decentralized regime, we introduce the *Verifiable Allocation Market POSG* (VAMP), a partially observable stochastic game in which agents coordinate through market-mediated incentives over subtasks and intermediate results.

5. We show that equilibrium computation in VAMPs is intractable in general, which motivates the search for approximate solution methods rather than exact ones.

6. We develop a multiagent reinforcement learning framework for approximating useful low-regret behavior in VAMPs, combining sequence-based policies, centralized training with decentralized execution, and population-based training.

7. We evaluate the resulting methods on synthetic task-allocation environments under several market mechanisms, and we instantiate the framework in neural theorem proving within Lean.

Chapter 2

Scheduling Theory and Task Allocation

With the motivation of modelling real-world mathematical development in mind, we are led to a general problem of allocating heterogeneous agents (i.e. mathematicians) across a large pool of interdependent preemptimble tasks (i.e. theorems). These agents operate in a decentralized manner, with tasks having a natural implicit dependency structure between them with agents able to generate subtasks (e.g., conjectures, lemmas) that may assist in the completion of other tasks. The goal is to maximize collective progress towards some target set of tasks (e.g., the proof of a major conjecture) in a minimal amount of time.

This problem of interdependent task allocation is closely related to classical scheduling theory, which studies how to allocate jobs to machines under various constraints and objectives. In the standard taxonomy of machine scheduling, one studies how jobs should be allocated to machines under objectives such as makespan or completion time, with increasingly general machine models ranging from identical to uniform to unrelated machines.

More fundamentally, one may consider the problem of scheduling along three orthogonal axes: deterministic vs. stochastic environment, offline vs. online environment, and deterministic vs. stochastic policy. We review the relevant scheduling theory along these axes, noting that deterministic environments are a special case of the more general stochastic environment, and that offline environments are a special case of the more general online environment, and deterministic policies are a special case of the more general stochastic policies.

2.1 Classical Scheduling Theory

2.1.1 Offline Deterministic Scheduling

Classical scheduling theory provides a mathematical foundation for understanding how to optimally allocate jobs to machines under various constraints. Its importance lies in offering formal models and algorithms for resource allocation problems that arise ubiquitously in

manufacturing, computing, project management, and logistics. By developing a taxonomy of machine environments, job characteristics, and objectives, scheduling theory enables the robust analysis and study of such resource allocation tasks.
More rigorously, given n jobs $\{J_j\}_{j=1}^n$ and m machines $\{M_i\}_{i=1}^m$, we may characterize offline deterministic scheduling problems by Graham, Lawler, Lenstra, and Rinnooy-Kan's three-field notation $\alpha \mid \beta \mid \gamma$, consisting of a machine environment α, a job environment β, and an objective γ.

Definition 2.1 (Machine Environment)**.** A machine environment $\alpha = \alpha_1\alpha_2$, given by $\alpha_1 \in \{\epsilon, P, Q, R, O, F, J\}, \alpha_2 \in \{\epsilon\} \cup \mathbb{N}$ is defined as follows:

- $\alpha_1 \in \{\epsilon, P, Q, R\}$: each J_j consists of a single operation processable on any M_i with processing time p_{ij} as follows:
  - $\alpha_1 = \epsilon$: single machine $p_{1j} = p_j$
  - $\alpha_1 = P$: identical parallel machines $p_{ij} = p_j$
  - $\alpha_1 = Q$: uniform parallel machines $p_{ij} = q_i p_j$ where q_i is the speed of machine M_i
  - $\alpha_1 = R$: unrelated parallel machines with arbitrary p_{ij}
- $\alpha_1 = O$: open shop with operation set $\{O_{1j}, \cdots, O_{mj}\}$ for each job J_j. O_{ij} must be processed on machine M_i in p_{ij} time, but the order is arbitrary., each operation processable on any machine with processing time p_{ij}.
- $\alpha_1 = J$: job shop with operations $(O_{1j}, \cdots, O_{m_j j})$ for each job J_j. O_{ij} must be processed on machine μ_{ij} in p_{ij} time, with $\mu_{i-1,j} \neq \mu_{ij}$ for $i = 2, \cdots, m_j$.
- $\alpha_1 = F$: flow shop with operations $(O_{1j}, \cdots, O_{mj})$ for each job J_j. O_{ij} must be processed on machine M_i in p_{ij} time, and the order is fixed such that O_{1j} must be processed before O_{2j}, which must be processed before O_{3j}, etc.

Additionally, if $\alpha_2 \in \mathbb{N}$, there are $m = \alpha_2$ machines; otherwise, m is assumed to be variable. Note, $\alpha_1 = \epsilon \iff \alpha_2 = 1$.

Definition 2.2 (Job Environment)**.** A job environment $\beta \subseteq \{\beta_1, \cdots, \beta_6\}$ is given as a collection of job characteristics, which are given as follows:

- $\beta_1 \in \{\text{pmtn}, \epsilon\}$: If $\beta_1 = \text{pmtn}$, preemption is allowed (an operation may be interrupted and resumed later). Otherwise, no preemption.

- $\beta_2 \in \{\text{res}, \text{res1}, \epsilon\}$: If $\beta_2 = \text{res}$, then s limited resources R_h are present; job J_j requires r_{hj} units of R_h throughout its processing, and resource capacities cannot be exceeded. If $\beta_2 = \text{res1}$, a single limited resource is present and otherwise for $\beta_2 = \epsilon$, there are no resource constraints.

- $\beta_3 \in \{\text{prec}, \text{tree}, \epsilon\}$: If $\beta_3 = \text{prec}$, then a precedence relation $\prec$ between jobs is defined via a DAG G with vertex set $[n]$. If there exists a directed path from j to k, then $J_j \prec J_k$ and J_j must complete before J_k starts.

  If $\beta_3 = \text{tree}$, we moreover require that G is a rooted tree. Otherwise, if $\beta_3 = \epsilon$, there are no precedence constraints.

- $\beta_4 \in \{r_j, \epsilon\}$: If $\beta_4 = r_j$, job-specific release dates are specified, and otherwise, all jobs are available at time 0.

- $\beta_5 \in \{m_j \leq \bar{m}, \epsilon\}$ (only relevant for $\alpha_1 = J$): If $\beta_5 = m_j \leq \bar{m}$, then a constant upper bound on the number of operations per job is specified. Otherwise, no bound on operations is specified.

- $\beta_6 \in \{p_{ij} = 1,\ p^- \leq p_{ij} \leq p^+,\ \epsilon\}$: If $\beta_6 = p_{ij} = 1$, all operation processing times are unit. Otherwise, if $\beta_6 = p^- \leq p_{ij} \leq p^+$, uniform lower/upper bounds on processing times are specified. If $\beta_6 = \epsilon$, we have no such bounds.

Definition 2.3 (Objective)**.** An objective $\gamma \in \{f_{\max}, f_{\sum}\}$ indicates the optimality criterion one is optimizing for. Namely, given a schedule, one may compute for each job J_j:

- total completion time C_j
- latency $L_j = C_j - d_j$
- tardiness $T_j = \max\{0, C_j - d_j\}$
- unit penalty $U_j = \begin{cases} 1 & \text{if } C_j > d_j \\ 0 & \text{otherwise} \end{cases}$

Where the latter three are well defined if due dates d_j are specified for each job. Then, if $\gamma = f_{\max}$, the objective is to minimize $f_{\max} \in \{C_{\max}, L_{\max}\}$, where $C_{\max} = \max_j C_j$ is the maximum completion time and $L_{\max} = \max_j L_j$ is the maximum latency.
Alternatively, if $\gamma = f_{\sum}$, the objective is to minimize

$$f_{\sum} \in \left\{ \sum_j C_j,\ \sum_j T_j,\ \sum_j U_j,\ \sum_j w_j C_j,\ \sum_j w_j T_j,\ \sum_j w_j U_j \right\}$$

for some weights w_j.

With this, we may naturally define an instance of the (offline) deterministic scheduling problem as:

Definition 2.4 (Offline Deterministic scheduling instance)**.** An offline deterministic scheduling instance I is given as a tuple:

$$I = (\mathcal{J}, \mathcal{M}, \alpha \mid \beta \mid \gamma)$$

Augmented with any additional data required by α, β, γ, such as processing times p_{ij}, precedence DAG G, job operations O_j, resource requirements r_{hj}, objective weights w_j, etc.
In practice, we say this is an (offline deterministic) instance of type $\alpha \mid \beta \mid \gamma$ with jobs $\mathcal{J} = \{J_1, \ldots, J_n\}$ and machines $\mathcal{M} = \{M_1, \ldots, M_m\}$, where the machine environment α, job environment β, and objective γ are given as above.

2.1.2 Offline Stochastic Scheduling

Real-world environments are often subject to various sources of uncertainty, such as variability in processing times, machine breakdowns, etc. To model such phenomena, one may extend the classical scheduling framework to incorporate stochastic primitives, leading to the study of optimal scheduling under uncertainty.
Towards this, we consider scheduling models in which the distributions of processing times, release dates, due dates, or other relevant parameters are known at time zero, however, we note that the actual outcome of a random processing time is known only at the time of its completion. Similarly, the realization of a release date or due date is known only at the point in time when it occurs.

Definition 2.5 (Offline Stochastic scheduling instance)**.** An (offline) stochastic scheduling instance I is given as a tuple:

$$I = (\mathcal{J}, \mathcal{M}, \alpha \mid \beta \mid \gamma, (\Omega, \mathcal{F}, \mathbb{P}))$$

Namely, we consider such an instance with job set $\mathcal{J} = \{J_1, \ldots, J_n\}$ and machines $\mathcal{M} = \{M_1, \ldots, M_m\}$ of type $\alpha \mid \beta \mid \gamma$ defined with respect to a probability space $(\Omega, \mathcal{F}, \mathbb{P})$. The machine environment α, job environment β, and objective γ are defined as in the deterministic case, but with primitives now allowed to be given as a random variable.
More rigorously, a stochastic scheduling instance of type $\alpha \mid \beta \mid \gamma$ is defined analogously to the deterministic case, but with the following modifications:

- processing times p_{ij} may be given as random variables $P_{ij} : \Omega \to (0, \infty)$
- if $\alpha_1 = Q$, machine speeds q_i may be given as random variables $Q_i : \Omega \to (0, \infty)$, and processing times are given by $P_{ij} = Q_i P_j$.
- if $\alpha_1 \in \{O, J, F\}$, for each job J_j, the processing time of each operation $O_{ij} \in \{O_{1j}, \ldots, O_{m_j j}\}$ may be given as a random variable $P_{ij} : \Omega \to (0, \infty)$.
- If $\beta_2 = \text{res}$, the resource requirements r_{hj} for a job J_j may be given as random variables $R_{hj} : \Omega \to [0, \infty)$.
- If $\beta_4 = r_j$, the release date of job J_j may be given as random variables $R_j : \Omega \to [0, \infty)$.
- If γ relies on due dates, the due date of job J_j may be given as random variables $D_j : \Omega \to [0, \infty)$.

Additionally, we extend the objective γ to stochastic analogues of the aforementioned objectives; in practice, this most commonly occurs through expectation: $\Gamma \in \{\mathbb{E}(f_{\max}), \mathbb{E}(f_{\sum})\}$. For example, one may consider the expected makespan objective $\Gamma = \mathbb{E}(C_{\max})$.

Remark (Latent realization versus observation). All primitive random variables in an offline stochastic instance are defined on the same probability space. Hence, for each $\omega \in \Omega$, the full collection of primitive parameters already has a realized value.

Remark (Observation convention). The bare instance definition specifies the latent random primitives, but not yet the scheduler's information process. In the standard stochastic scheduling convention, the distributions of the primitive random variables are known in advance, while their realized values are only observed when they become operationally relevant. Typical conventions are:

- a realized processing requirement is observed upon completion of the corresponding job or operation;
- a realized release date is observed when the job is released;
- a realized due date is either known at time 0 or else observed when it becomes relevant, depending on the model;
- any other random primitive is observed according to the information convention specified later when defining states and policies.

In this setting, the scheduler must make decisions based on the information available at each point in time, without knowledge of future random outcomes. Indeed, our objective in stochastic scheduling is not simply to design algorithms that optimize our desired objective γ, but rather to do so in a nonanticipative manner. This will be formalized in more rigor and generality in the following section.

2.1.3 Stochastic Online Scheduling

In online scheduling, the defining feature is not merely uncertainty, but sequential revelation of the instance. The scheduler does not know the full instance at time 0; instead, jobs, operations, precedence constraints, or other problem data are revealed over time, and decisions must be made using only the information currently available. In the fully stochastic regime, the unrevealed portion of the instance may itself be random. Here, we consider the more general stochastic online scheduling setting, which subsumes the deterministic online scheduling setting as a special case.

Definition 2.6 (Stochastic online scheduling instance)**. TODO: OOPSIE FORGOT TO TALK ABOUT PRECEDENCE DAG!!!** A stochastic online scheduling instance I is given by a tuple

$$I = (\mathcal{J}^*, \mathcal{M}, \alpha \mid \beta \mid \gamma, (\Omega, \mathcal{F}, \mathbb{P}), \mathcal{V}),$$

Namely, we consider an instance with a (perhaps countable) latent job set $\mathcal{J}^*$ and machine set $\mathcal{M} = \{M_1, \ldots, M_m\}$ of type $\alpha \mid \beta \mid \gamma$ defined with respect to a probability space $(\Omega, \mathcal{F}, \mathbb{P})$. Here, we additionally include a family of revelation-time random variables $\mathcal{V} = (V_j)_{J_j \in \mathcal{J}^*}$, where $V_j : \Omega \to [0, \infty)$ denotes the time at which job J_j becomes visible to the scheduler.
As in the offline stochastic case, the numerical primitives attached to each latent job may be random variables on $(\Omega, \mathcal{F}, \mathbb{P})$. In particular, consider the random variables:

- processing requirements $P_{ij} : \Omega \to (0, \infty)$ or $P_j : \Omega \to (0, \infty)$;
- if $\alpha_1 = Q$, machine speeds $Q_i : \Omega \to (0, \infty)$;
- if $\alpha_1 \in \{O, J, F\}$, operation processing requirements $P_{ij} : \Omega \to (0, \infty)$;
- if $\beta_2 = \text{res}$, resource requirements $R_{hj} : \Omega \to [0, \infty)$;
- if $\beta_4 = r_j$, release dates $R_j : \Omega \to [0, \infty)$. We distinguish between release date and revelation times for the sake of comprehensiveness, only requiring $R_j \geq V_j$. However, in practice, one often assumes $R_j = V_j$;

- if γ depends on due dates, due dates $D_j : \Omega \to [0, \infty)$.

The defining feature of the online model is that the full latent job set $\mathcal{J}^*$ and its associated primitive data need not be visible to the scheduler at time 0. Instead, a job J_j becomes available for consideration only when it is revealed at time V_j.

Remark (Revealed job set). For each $t \geq 0$ and $\omega \in \Omega$, the set of jobs visible to the scheduler by time t is

$$\mathcal{J}_t(\omega) := \{J_j \in \mathcal{J}^* : V_j(\omega) \leq t\}.$$

Thus $(\mathcal{J}_t)_{t \geq 0}$ is the revealed-job process induced by the latent instance and the revelation times V_j.

Remark (Observation convention). Unless otherwise specified, when a job J_j is revealed at time V_j, the scheduler learns the existence of J_j together with any job-level metadata designated as visible upon revelation. In the simplest convention, this includes data such as due dates, release dates, resource requirements, or the distributions of latent processing requirements.

As in the offline stochastic case, the realized processing requirement itself need not be observed upon revelation. Instead, it may remain latent and become observable only when the corresponding job or operation completes.

With this, we may simply define the deterministic online scheduling setting as a special case of the stochastic online scheduling setting:

Definition 2.7 (Deterministic online scheduling instance)**.** A deterministic online scheduling instance is a stochastic online scheduling instance in which all primitives are deterministic, i.e. all random variables are degenerate.

2.1.4 States, Actions, and Policies

Fix a stochastic online scheduling instance

$$I = (\mathcal{J}^*, \mathcal{M}, \alpha \mid \beta \mid \gamma, (\Omega, \mathcal{F}, \mathbb{P}), \mathcal{V}),$$

where $\mathcal{J}^*$ is the latent job set and $V_j : \Omega \to [0, \infty]$ is the revelation time of job $J_j \in \mathcal{J}^*$. Let $\mathcal{O}^*$ denote the latent operation set induced by the machine environment α on $\mathcal{J}^*$.

Definition 2.8 (Revealed state space)**.** For each $t \geq 0$ and $\omega \in \Omega$, define the revealed job set

$$\mathcal{J}_t(\omega) := \{J_j \in \mathcal{J}^* : V_j(\omega) \leq t\},$$

and let $\mathcal{O}_t(\omega) \subseteq \mathcal{O}^*$ denote the set of operations belonging to revealed jobs.
A revealed state for the instance I is a tuple

$$x = (\mathcal{J}^x, \mathcal{O}^x, \mathcal{E}^x, \mathcal{B}^x, \mathcal{C}^x, Y^x, \zeta^x),$$

where:

- $\mathcal{J}^x \subseteq \mathcal{J}^*$ is the set of jobs currently revealed;
- $\mathcal{O}^x \subseteq \mathcal{O}^*$ is the set of operations currently revealed;
- $\mathcal{E}^x \subseteq \mathcal{O}^x$ is the set of revealed operations that are currently eligible for processing, i.e. revealed, not completed, and currently admissible with respect to release constraints, precedence constraints, machine availability, and any other revealed feasibility constraints;
- $\mathcal{B}^x : \mathcal{M} \to \mathcal{O}^* \cup \{\varnothing\}$ records which operation, if any, is currently assigned to each machine;
- $\mathcal{C}^x \subseteq \mathcal{O}^x$ is the set of completed revealed operations;
- $Y^x : \mathcal{O}^* \to [0, \infty)$ records cumulative processing allocated so far to each operation, with $Y^x(o) = 0$ for $o \notin \mathcal{O}^x$;
- ζ^x records all other scheduler-visible data at the current time, such as revealed weights, due dates, release dates, revealed precedence arcs, machine states, resource states, processing-time distributions, or other primitive data specified by the observation convention of the instance.

The revealed state space X_I is the collection of all such tuples satisfying the feasibility constraints encoded by the instance I. We endow X_I with the sigma-algebra $\mathcal{X}_I$ generated by the coordinate maps.

Definition 2.9 (Action space and admissible actions)**.** Fix a stochastic online scheduling instance I with revealed state space $(\mathsf{X}_I, \mathcal{X}_I)$.
The *action space* $(\mathsf{A}_I, \mathcal{A}_I)$ is a measurable space whose elements encode instantaneous scheduling decisions, namely which revealed eligible operations are started, continued, resumed, preempted, or left idle on the machines.
For each state $x \in \mathsf{X}_I$, let

$$\mathcal{U}_I(x) \subseteq \mathsf{A}_I$$

denote the set of admissible actions at state x, i.e. those actions consistent with:

- machine eligibility constraints from α,
- job and operation constraints from β,
- the currently revealed information encoded in x,
- and the preemption convention of the instance.

In particular, an admissible action may refer only to revealed operations, and only to operations that are currently eligible in the revealed state x.

Definition 2.10 (Observed history)**.** For each $t \geq 0$, let H_t^I denote the measurable space of feasible revealed-state histories up to time t. An element $h \in \mathsf{H}_t^I$ is a path

$$h = (x_s)_{0 \leq s \leq t}$$

with $x_s \in \mathsf{X}_I$ for all $s \leq t$, consistent with the revelation and feasibility conventions of the instance I. We write $x_t(h)$ for the terminal state of the history h.
In discrete time, one may simply take

$$\mathsf{H}_t^I = \mathsf{X}_I^{t+1}$$

with the product sigma-algebra.

Definition 2.11 (Deterministic nonanticipative policy)**.** A *deterministic nonanticipative policy* for a stochastic online scheduling instance I is a family of measurable maps

$$\varphi_t : \mathsf{H}_t^I \to \mathsf{A}_I, \qquad t \geq 0,$$

such that

$$\varphi_t(h) \in \mathcal{U}_I(x_t(h)) \qquad \text{for every } h \in \mathsf{H}_t^I.$$

Thus, at time t, the action selected by the policy may depend only on the revealed history up to time t, and not on unrevealed future jobs or unrevealed future primitive values.

Definition 2.12 (Randomized nonanticipative policy)**. TODO: NOTE THIS IS EQUIVALENT TO A STOCH PROCESS ADAPTED TO FILTRATION GENERATED BY REVEALED HISTORY; ALSO COMPARE STOCHASTIC VS RANDOMIZED POLICY!!!** A randomized nonanticipative policy for a stochastic online scheduling instance I is a family of stochastic kernels

$$K_t(\cdot \mid h) \in \mathcal{P}(\mathsf{A}_I), \qquad t \geq 0,\ h \in \mathsf{H}_t^I,$$

such that, for every $h \in \mathsf{H}_t^I$,

$$K_t(\mathcal{U}_I(x_t(h)) \mid h) = 1.$$

Thus, conditional on the revealed history up to time t, the policy selects a probability distribution supported on the admissible actions at the current state.

Remark. A deterministic nonanticipative policy is a special case of a randomized nonanticipative policy, obtained by taking each kernel $K_t(\cdot \mid h)$ to be a Dirac mass at a single admissible action.

Remark (Restriction to offline stochastic scheduling). Suppose that every latent job is revealed at time 0, i.e.

$$V_j(\omega) = 0 \qquad \text{for all } J_j \in \mathcal{J}^* \text{ and all } \omega \in \Omega.$$

Then the revealed job set is constant:

$$\mathcal{J}_t(\omega) = \mathcal{J}^* \qquad \text{for all } t \geq 0.$$

Hence the above revealed-state, action, and policy definitions reduce to the corresponding notions for offline stochastic scheduling: the job and operation structure is fully known from time 0, while only the realized primitive values remain progressively observable according to the observation convention.

Remark (Restriction to online deterministic scheduling). Suppose that every primitive random variable in the instance is almost surely constant, while the revelation times V_j may remain nontrivial. Then the latent environment is deterministic, but not fully visible at time 0. The above definitions therefore reduce to the usual online deterministic model: the state records only the currently revealed part of the deterministic instance, and policies are nonanticipative with respect to the revealed history.

Remark (Restriction to offline deterministic scheduling). If, in addition to the previous restriction, every job is revealed at time 0, then both the primitive data and the revelation pattern are deterministic and fully known in advance. In that case the above definitions reduce to the ordinary offline deterministic scheduling model.

2.2 Examples

2.2.1 Offline deterministic: $P \mid \text{prec} \mid C_{\max}$

Consider the identical-parallel-machine makespan problem with precedence constraints. Here, we have n jobs with processing times p_j and precedence constraints given by a DAG G, and the goal is to schedule the jobs on m identical machines to minimize the makespan $C_{\max}$.

Graham's list scheduling algorithm gives a $(2 - \frac{1}{m})$-approximation for this problem on m identical machines. The problem is NP-hard in general; even special cases such as $P2 \mid$ chains $\mid C_{\max}$ are weakly NP-hard, and $P2 \mid \text{prec}, , p_j \in 1, 2 \mid C_{\max}$ is $W[2]$-hard when parameterized by partial-order width. On the positive side, for unit-size jobs on a fixed number of machines, a quasi-polynomial time approximation scheme (QPTAS) is known, and for $m = 3$, the exact complexity of the unit-size case remains famously delicate.

A nice way to present the algorithm is this. Fix any linear extension L of the precedence order. Whenever a machine becomes idle, schedule the first available job in L. Two canonical lower bounds on the optimal makespan are

$$\frac{1}{m}\sum_j p_j \qquad \text{and} \qquad \ell_{\max},$$

where $\ell_{\max}$ is the length of a longest precedence chain. If j is the last job completed by list scheduling, then the interval before j starts can be decomposed into busy periods and forced-idle periods. Busy periods contribute at most $\frac{1}{m}\sum_i p_i - \frac{p_j}{m}$, while forced-idle periods can be charged to a chain ending at j, contributing at most $\ell_{\max} - p_j$. Hence

$$C_{\max}^{LS} \le \frac{1}{m}\sum_i p_i + \Big(1 - \frac{1}{m}\Big)\ell_{\max} \le \Big(2 - \frac{1}{m}\Big)\text{OPT}.$$

2.2.2 Offline stochastic: $P \mid \text{prec} \mid \mathbb{E}[C_{\max}]$, and the weighted-completion extension

For the stochastic offline case, consider the stochastic counterpart of the same model as before. Here, Graham's list scheduling bound extends directly from deterministic makespan to expected makespan, namely, $P \mid \text{prec} \mid \mathbb{E}[C_{\max}]$ still admits a $(2 - \frac{1}{m})$-approximation. We do note, however, Skutella and Uetz show that once one moves from makespan to expected weighted completion time with precedence constraints, the problem becomes much subtler: ordinary Graham-style list scheduling can be arbitrarily bad, and LP-based delayed list scheduling is needed to recover constant-factor guarantees. Their paper gives the first constant-factor approximation algorithms for stochastic parallel-machine scheduling with precedence constraints.

The clean proof sketch for the makespan version is pathwise. For each realization ω, let

$$W(\omega) = \sum_j P_j(\omega), \qquad H(\omega) = \text{length of a longest realized precedence chain.}$$

Running the same list-scheduling rule as above gives, for each ω,

$$C_{\max}^{LS}(\omega) \le \frac{W(\omega)}{m} + \Big(1 - \frac{1}{m}\Big)H(\omega).$$

Since every feasible schedule must satisfy

$$\mathrm{OPT}(\omega) \geq \max\Big\{\frac{W(\omega)}{m}, , H(\omega)\Big\},$$

taking expectations yields

$$\mathbb{E}[C_{\max}^{LS}] \leq \Big(2 - \frac{1}{m}\Big)\mathbb{E}[\mathrm{OPT}],$$

and the stochastic problem is NP-hard already because the deterministic case is its degenerate special case.

In a related note, for

$$P \mid \mathrm{prec} \mid \mathbb{E}\left[\sum_j w_j C_j\right],$$

Skutella and Uetz combine an LP relaxation with delayed list scheduling to obtain constant-factor approximations; reporting a guarantee of

$$3 + 2\sqrt{2} - \frac{1+\sqrt{2}}{m}$$

for the precedence-constrained case. Indeed, once the objective changes from makespan to weighted completion, stochastic scheduling rapidly stops being a straightforward extension of the deterministic theory.

2.2.3 Stochastic online: classical SOS for $P \mid r_j \mid \mathbb{E}[\sum_j w_j C_j]$

Consider a problem where jobs arrive over time, on arrival the scheduler learns the job's weight and expected processing time, but the realized processing time remains unknown until completion. In this model, the number of jobs is not known in advance, jobs arrive online in release-date order, and the objective is expected weighted completion time. When release dates are absent on a single machine, weighted shortest expected processing time (WSEPT) is the natural and well-known rule, namely, order jobs based on the highest ratio of weight to expected processing time. However, with release dates, we can consider the α-Shift-WSEPT policy. Moreover, on parallel machines, one may combine it with fixed-assignment policies such as Modified MinIncrease or RandAssign.

On one machine, α-Shift-WSEPT modifies each release date to

$$r'_j = \max r_j, \alpha, \mathbb{E}[P_j]$$

and then always runs the available job of maximum Smith ratio $w_j/\mathbb{E}[P_j]$. Megow, Uetz, and Vredeveld prove that this is a $(\delta + 2)$-approximation for

$$1 \mid r_j \mid \mathbb{E}\left[\sum_j w_j C_j\right]$$

when processing times are δ-NBUE, with the best choice $\alpha = 1$. For the parallel-machine version

$$P \mid r_j \mid \mathbb{E}\left[\sum_j w_j C_j\right]$$

This modified MinIncrease policy assigns each arriving job to the machine minimizing an incremental expected-cost expression and then sequences jobs on that machine by α-Shift-WSEPT. This gives the bound

$$\rho = 1 + \max\left\{1 + \frac{\delta}{\alpha}, \alpha + \delta + \frac{(m-1)(\Delta+1)}{2m}\right\},$$

where Δ bounds the squared coefficient of variation. For NBUE processing times, optimizing α gives a ratio below $3.62 - \frac{1}{2m}$. Their randomized policy RandAssign, which assigns each revealed job uniformly at random to a machine and then runs α-Shift-WSEPT locally, achieves the same bound.

This example also gives a clean online lower-bound. Because deterministic processing times are contained as a special case, classical deterministic online lower bounds transfer immediately to SOS. Megow, Uetz, and Vredeveld recall that in the deterministic single-machine online-time model, a 2-competitive algorithm exists and this lower bound is tight; for identical parallel machines, they cite a lower bound of 1.309 for deterministic online algorithms, a best-known deterministic upper bound of 2.62, and a randomized upper bound of 2. They also prove lower bounds internal to their SOS framework: any fixed-assignment policy has asymptotic lower bound about 1.24, and their MinIncrease policy has lower bound 3/2 for general distributions.

2.3 Extensions and Generalizations

The classical SOS model above assumes that jobs arrive exogenously over time, that each revealed job comes with fully visible distributional metadata at realization time, that precedence information is fixed and revealed in a simple manner, and that each job is completed only once all of its operations are completed. Towards the development of the centralized ITAP setting in future sections, we extend this online stochastic formalism such that latent primitives may be only partially observed, may depend on the currently revealed history, and may permit richer completion and resource-allocation semantics.

Definition 2.13 (Belief-augmented generalized stochastic online scheduling instance)**.** A belief-augmented generalized stochastic online scheduling instance consists of:

1. a baseline stochastic online scheduling instance

$$I_0 = (\mathcal{J}^*, \mathcal{M}, \alpha \mid \beta \mid \gamma, (\Omega, \mathcal{F}, \mathbb{P}), \mathcal{V}),$$

   as defined above;

2. a measurable latent parameter space $(\Theta, \mathcal{T})$ and a latent random element

$$\theta : \Omega \to \Theta,$$

   where θ encodes the hidden primitive data of the environment, such as latent processing laws, latent precedence information, latent tractability of operations, or other scheduler-relevant unknowns;

3. for each revealed history h and each revealed operation o, a measurable processing-law kernel

$$\kappa_o^{\mathrm{proc}}(\,\cdot\,\mid h, \theta) \in \mathcal{P}((0, \infty]),$$

   representing the conditional law of the processing requirement of o given the current revealed history and latent parameter θ;

4. a latent precedence structure, represented for example by a latent DAG

$$G^\theta = (\mathcal{J}^*, E^\theta),$$

   together with a revealed-precedence process

$$G_t = (\mathcal{J}_t, E_t), \qquad E_t \subseteq E^\theta \cap (\mathcal{J}_t \times \mathcal{J}_t),$$

   specifying which precedence relations have become visible by time t;

5. for each job J_j, a completion family

$$\mathcal{W}_j \subseteq 2^{\mathcal{O}_j},$$

   where $\mathcal{O}_j$ is the operation set of J_j, and the job is declared complete once the set of completed operations contains some $W \in \mathcal{W}_j$;

6. a redundancy/progress rule specifying, for each revealed history h, latent parameter θ, operation o, and subset of machines $S \subseteq \mathcal{M}$, the rate or law by which cumulative processing accrues when the machines in S are simultaneously assigned to o.

Definition 2.14 (Belief state)**.** Fix a belief-augmented generalized stochastic online scheduling instance and let H_t denote the revealed history up to time t. The corresponding belief state at time t is the posterior law

$$\Pi_t := \mathrm{Law}(\theta \mid H_t).$$

The belief-augmented state is then

$$\widehat{X}_t := (X_t, \Pi_t),$$

where X_t is the revealed state from the baseline SOS formalism.

Remark (State-dependent processing laws)*.* The generalized formalism permits the processing law of a revealed operation to depend on the currently completed set. This captures situations in which completing some jobs changes the effective difficulty of future jobs, either by unlocking information, enabling reuse of intermediate results, or modifying the latent environment.

Remark (Stochastic and progressively revealed precedence)*.* The generalized formalism permits the latent precedence relation itself to be random and only progressively revealed. A particularly relevant convention is that precedence information involving a job is revealed only upon completion of that job, so that the revealed-precedence process G_t grows at completion times.

Remark (Noisy priors and partial distributional information)*.* In the generalized model, the scheduler need not observe the full processing-time law or other primitive distributions at job revelation. Instead, revelation may provide only partial information, and the scheduler must act on the posterior belief Π_t. Classical SOS is recovered as the special case in which the full ex ante processing-time distribution of each revealed job is visible upon revelation.

Remark (Redundant scheduling)*.* The generalized formalism permits redundant scheduling, in which multiple machines may simultaneously process the same revealed operation. The effect of such assignments is encoded by the redundancy/progress rule. The classical nonredundant model is recovered by imposing that at most one machine may be assigned to any operation at a given time.

Remark (Generalized completion semantics)*.* The completion family $\mathcal{W}_j \subseteq 2^{\mathcal{O}_j}$ permits completion semantics more general than the usual shop models. The classical AND-case is recovered by taking

$$\mathcal{W}_j = \{\mathcal{O}_j\},$$

so that all operations of J_j must be completed. An OR-shop is obtained by taking

$$\mathcal{W}_j = \big\{\{o\} : o \in \mathcal{O}_j\big\},$$

so that completing any one operation suffices to complete the job.
Note additionally that the generalized processing-law kernels may assign infinite processing time to some operations, thereby modeling intractable alternatives. In the OR-shop setting, one may consider such intractible operations, assuming each job J_j has at least one operation in $\mathcal{O}_j$ is almost surely tractable, and study allocation under uncertainty about which operations are tractable.

Taken together, the above extensions yield an online stochastic scheduling formalism in which:

- the effective difficulty of a task may depend on the currently completed set;
- the dependency structure itself may be latent and progressively revealed;
- the scheduler may act under partial distributional information and update beliefs over time;
- multiple machines may collaborate redundantly on a single task;
- and a job may admit multiple alternative completion routes.

This is precisely the level of generality needed for the centralized ITAP model.

Chapter 3

The Interdependent Task Allocation Problem

3.1 Centralized ITAP

The scheduling models of the previous chapter provide a useful language for allocation under uncertainty, partial revelation, and precedence structure. However, the research-allocation problems motivating this thesis are not fully captured by classical scheduling alone. In mathematical development, software verification, and related domains, tasks do not merely wait to be processed: they may reveal further tasks, alter the effective difficulty of future work, admit multiple alternative solution routes, and interact with heterogeneous agents in strongly state-dependent ways. The purpose of this section is to formalize a centralized benchmark model that captures these phenomena while retaining the operational flavor of stochastic online scheduling. This model is the *Interdependent Task Allocation Problem* (ITAP) in its centralized regime.

3.1.1 Motivation

The motivating picture is the one already suggested in the introduction: major intellectual achievements rarely arise from a single isolated task solved in a vacuum. Rather, they emerge from the coordinated resolution of many interdependent tasks, some visible from the outset and others only becoming apparent once earlier work has been completed. In mathematics, this is especially clear. A target theorem may depend on a large body of intermediate lemmas, conceptual reorganizations, and auxiliary infrastructure; some of these dependencies are known in advance, while others are only discovered as the work progresses. More broadly, the same pattern appears in theorem proving, formal verification, experimental science, and engineering design: progress is not merely the execution of a fixed list of jobs, but the navigation of an evolving dependency structure over tasks whose values are largely

downstream.

Classical scheduling captures part of this story. It allows one to reason about heterogeneous resources, release dates, precedence constraints, preemption, and stochastic processing times. Yet it still treats jobs as largely exogenous objects whose significance is intrinsic and whose role in the future of the system is secondary. By contrast, in the settings motivating this thesis, the importance of a task is often determined less by its immediate completion and more by what its completion reveals, enables, or disambiguates. Solving a task may unlock new tasks, certify that some alternative routes are unnecessary, change the effective tractability of other tasks, or expose previously latent structure in the dependency graph itself.

This motivates viewing research and related forms of problem-solving as an *interdependent* allocation problem. We imagine a central planner allocating a collection of heterogeneous agents across a pool of partially revealed tasks. Each task may admit several possible modes of attack, several agents may collaborate on the same task, and the time required to complete a task may depend on both the current state of completed work and latent environmental factors that are not fully visible to the planner. At the same time, completing tasks changes the future by revealing new tasks and by modifying the set of feasible or promising continuations.

The centralized regime is the analytically clean starting point for this study. In this regime there is a single planner, a single objective, and no private information or strategic behavior among agents. Thus the centralized ITAP should be understood as a controlled, belief-augmented generalization of the stochastic online scheduling models introduced earlier. It serves as both a benchmark model in its own right and as the conceptual bridge to the decentralized regime considered later, where private information and strategic interaction can no longer be ignored.

3.1.2 Formal Definition

We now define the centralized ITAP as a particular belief-augmented generalized stochastic online scheduling instance. Throughout this section, we work in discrete time $t \in \mathbb{N}_0$, in keeping with the discrete-time focus adopted earlier.

Definition 3.15 (Centralized Interdependent Task Allocation Problem)**.** A *centralized Interdependent Task Allocation Problem* (centralized ITAP) instance is a tuple

$$\mathfrak{I} = \big(\mathcal{A},\, \mathcal{T}^*,\, \{\mathcal{O}_\tau\}_{\tau\in\mathcal{T}^*},\, \{\mathcal{W}_\tau\}_{\tau\in\mathcal{T}^*},\, (\Omega, \mathcal{F}, \mathbb{P}),\, (\Theta, \mathcal{T}, \theta),\, \mathfrak{R},\, \{L_{\tau,o}\}_{\tau\in\mathcal{T}^*,\ o\in\mathcal{O}_\tau},\, \{\rho_{\tau,o}\}_{\tau\in\mathcal{T}^*,\ o\in\mathcal{O}_\tau},\, \mathfrak{Z},\, \Gamma\big),$$

consisting of the following data.

1. **Agents.** A finite set of agents

$$\mathcal{A} = \{a_1, \ldots, a_m\}.$$

These are the centralized analogue of the machines in the preceding scheduling models, but we use the term "agent" to emphasize that they may represent mathematicians, theorem provers, verification engines, experimental resources, or other heterogeneous workers.

2. **Latent task set.** A finite or countable latent task set

$$\mathcal{T}^*.$$

Elements $\tau \in \mathcal{T}^*$ are the tasks that may in principle arise during the execution of the system. The full latent task set need not be visible at time 0.

3. **Modes of attack / operations.** For each task $\tau \in \mathcal{T}^*$, a finite nonempty set

$$\mathcal{O}_\tau$$

of operations or modes of attack for task τ. Intuitively, these encode different ways of attempting the task. In the simplest case one may take $|\mathcal{O}_\tau| = 1$ for every τ, but the general formalism permits multiple alternative routes.

4. **Completion semantics.** For each task $\tau \in \mathcal{T}^*$, a nonempty family

$$\mathcal{W}_\tau \subseteq 2^{\mathcal{O}_\tau} \setminus \{\varnothing\}.$$

A task τ is considered complete once the set of completed operations for τ contains some $W \in \mathcal{W}_\tau$. Thus:

- the classical "AND" case is recovered by taking $\mathcal{W}_\tau = \{\mathcal{O}_\tau\}$;
- the classical "OR" case is recovered by taking

$$\mathcal{W}_\tau = \big\{\{o\} : o \in \mathcal{O}_\tau\big\}.$$

5. **Latent environment and prior.** A probability space

$$(\Omega, \mathcal{F}, \mathbb{P})$$

together with a measurable latent parameter space $(\Theta, \mathcal{T})$ and a measurable random element

$$\theta : \Omega \to \Theta.$$

The latent parameter $\theta(\omega)$ encodes all hidden environmental features relevant to the instance, such as latent difficulty classes, latent dependency information, latent tractability of operations, or any other hidden primitive data. The prior law of θ under $\mathbb{P}$ is the planner's ex ante belief.

6. **Task revelation operator.** A measurable map

$$\mathfrak{R} : 2^{\mathcal{T}^*} \times \Theta \to 2^{\mathcal{T}^*}$$

such that for every $\vartheta \in \Theta$:

- $\mathfrak{R}(C, \vartheta) \supseteq C$ for all $C \subseteq \mathcal{T}^*$;
- if $C \subseteq C'$, then

$$\mathfrak{R}(C, \vartheta) \subseteq \mathfrak{R}(C', \vartheta).$$

Thus $\mathfrak{R}(C, \vartheta)$ is the set of tasks revealed once the currently completed task set is C and the latent environment is ϑ. The monotonicity assumption formalizes the idea that completing more tasks cannot erase previously revealed tasks.

We write

$$\mathcal{T}_0^{\vartheta} := \mathfrak{R}(\varnothing, \vartheta)$$

for the initially revealed task set under latent environment ϑ.

7. **Latent workloads.** For each task $\tau \in \mathcal{T}^*$ and operation $o \in \mathcal{O}_\tau$, a latent workload random variable

$$L_{\tau,o} : \Omega \to (0, \infty].$$

Here $L_{\tau,o}(\omega)$ is the total amount of effective work required to complete operation o of task τ in scenario ω. The value $+\infty$ is allowed, thereby permitting intractable operations.

8. **State-dependent productivity / progress rule.** For each task $\tau \in \mathcal{T}^*$ and operation $o \in \mathcal{O}_\tau$, a measurable map

$$\rho_{\tau,o} : 2^{\mathcal{A}} \times 2^{\mathcal{T}^*} \times \Theta \to [0, \infty),$$

where

$$\rho_{\tau,o}(S, C, \vartheta)$$

is the instantaneous effective work rate on operation o when the set of agents $S \subseteq \mathcal{A}$ is simultaneously assigned to o, the completed task set is C, and the latent environment is ϑ.

This map encodes:

- heterogeneity of agent capabilities;
- redundancy or collaboration, since $|S|$ may exceed 1;
- state dependence of task difficulty through the dependence on C.

We require $\rho_{\tau,o}(\varnothing, C, \vartheta) = 0$ for all τ, o, C, ϑ.

9. **Visible signals / observation rule.** A family $\mathfrak{Z}$ of measurable signals specifying what the centralized planner observes when tasks are revealed and when operations complete. Concretely, this includes at least:

   - for each task τ, a revelation signal describing the metadata that become visible when τ enters the revealed task set;
   - for each operation (τ, o), a completion signal describing the information that becomes visible when o completes.

   In particular, the formalism allows that the planner may observe only partial information about the latent workload law or dependency structure at task revelation time, and must therefore act on a posterior belief rather than on full distributional knowledge.

10. **Objective functional.** A measurable performance functional

$$\Gamma$$

acting on complete trajectories of the controlled process. Depending on the application, Γ may minimize time to complete a target set, maximize expected cumulative value of completed tasks, minimize weighted completion times, or optimize any other measurable functional of the task-allocation trajectory.

Remark (Interpretation of the revelation operator). The revelation operator $\mathfrak{R}$ is the formal device by which the centralized ITAP captures recursive task generation. A task need not

be visible from the outset. Instead, it may become visible only after certain other tasks have been completed. One may equivalently think of $\mathfrak{R}$ as being induced by a latent dependency hypergraph, but for the centralized ITAP it is convenient to work directly with the monotone revelation operator, since it captures both ordinary precedence constraints and more general completion-triggered task revelation.

Remark (Interpretation of state-dependent productivity). The maps $\rho_{\tau,o}$ encode the fact that the effective difficulty of a task may depend on the currently completed set. This is the key point at which the ITAP departs from ordinary scheduling. Completing one task may simplify another, render a previously ambiguous path straightforward, or make an alternative route unnecessary. Thus task difficulty is not treated as a fixed exogenous label, but as a state-dependent quantity.

Remark (Beliefs and partial visibility). The latent parameter θ and the observation family $\mathfrak{Z}$ together encode the planner's partial knowledge. At any time t, the centralized planner need not know the full latent environment, nor even the full laws of the unresolved tasks. Instead, it acts on the posterior belief induced by the revealed history. Classical stochastic online scheduling is recovered as the special case in which, upon task revelation, the relevant processing-time distribution is fully visible.

We now define the induced state, action, and policy objects for a fixed centralized ITAP instance.

Definition 3.16 (Centralized ITAP state)**.** Fix a centralized ITAP instance $\mathfrak{I}$. A state at time t is a tuple

$$X_t = \big(\mathcal{T}_t,\, \mathcal{C}_t,\, \{\mathcal{C}^{\mathrm{op}}_{\tau,t}\}_{\tau\in\mathcal{T}_t},\, B_t,\, Y_t,\, \zeta_t,\, \Pi_t\big),$$

where:

- $\mathcal{T}_t \subseteq \mathcal{T}^*$ is the set of tasks revealed by time t;
- $\mathcal{C}_t \subseteq \mathcal{T}_t$ is the set of tasks completed by time t;
- for each revealed task $\tau \in \mathcal{T}_t$,

  $$\mathcal{C}^{\mathrm{op}}_{\tau,t} \subseteq \mathcal{O}_\tau$$

  is the set of completed operations of τ by time t;
- $$B_t : \mathcal{A} \to \Big(\bigsqcup_{\tau\in\mathcal{T}_t} \mathcal{O}_\tau\Big) \cup \{\varnothing\}$$

records the current centralized assignment of each agent, with $B_t(a) = \varnothing$ meaning that agent a is idle;

- $$Y_t(\tau, o) \in [0, \infty)$$

  is the cumulative effective work accrued on operation $o \in \mathcal{O}_\tau$ up to time t;

- ζ_t records all scheduler-visible metadata and signals revealed by time t;

- $$\Pi_t := \mathrm{Law}(\theta \mid H_t)$$

  is the posterior belief over latent environments given the revealed history H_t up to time t.

These data are required to satisfy:

1. $\mathcal{T}_t = \mathfrak{R}(\mathcal{C}_t, \theta)$ at the latent level, with only the visible portion represented in ζ_t;

2. a task $\tau \in \mathcal{T}_t$ is complete at time t if and only if there exists $W \in \mathcal{W}_\tau$ such that

$$W \subseteq \mathcal{C}^{\mathrm{op}}_{\tau,t};$$

3. an operation $o \in \mathcal{O}_\tau$ is complete once its cumulative effective work reaches its latent workload:

$$Y_t(\tau, o) \geq L_{\tau,o}(\omega).$$

Definition 3.17 (Centralized admissible actions)**.** Fix a centralized ITAP instance $\mathfrak{I}$ and a state x. An admissible action at x is a map

$$u : \mathcal{A} \to \Big(\bigsqcup_{\tau \in \mathcal{T}^x} \mathcal{O}_\tau \Big) \cup \{\varnothing\}$$

such that:

- if $u(a) = (\tau, o)$, then $\tau \in \mathcal{T}^x \setminus \mathcal{C}^x$;

- the operation $o \in \mathcal{O}_\tau$ has not already been completed;

- any additional revealed feasibility constraints encoded in x are respected.

Multiple agents may be assigned to the same operation simultaneously. Thus centralized actions are global assignment vectors, chosen by a single planner, and not independent local actions of the agents.

Definition 3.18 (Centralized nonanticipative policy)**.** For each $t \geq 0$, let $\mathsf{H}_t^{\mathfrak{I}}$ denote the measurable space of feasible histories of centralized ITAP states up to time t. A *deterministic centralized nonanticipative policy* is a family of measurable maps

$$\varphi_t : \mathsf{H}_t^{\mathfrak{I}} \to \mathsf{U}_{\mathfrak{I}}, \qquad t \in \mathbb{N}_0,$$

such that

$$\varphi_t(h) \in \mathcal{U}_{\mathfrak{I}}(x_t(h)) \qquad \text{for every } h \in \mathsf{H}_t^{\mathfrak{I}},$$

where $x_t(h)$ is the terminal state of the history h, $\mathsf{U}_{\mathfrak{I}}$ is the global action space, and $\mathcal{U}_{\mathfrak{I}}(x)$ is the admissible-action set at state x.
A *randomized centralized nonanticipative policy* is a family of stochastic kernels

$$K_t(\cdot \mid h) \in \mathcal{P}(\mathsf{U}_{\mathfrak{I}})$$

supported on $\mathcal{U}_{\mathfrak{I}}(x_t(h))$.
Thus centralized policies are precisely the global, history-dependent allocation rules by which a single planner chooses, at each time, how all agents are assigned across the currently revealed tasks.

Remark (Centralized ITAP as a specialization of generalized SOS)*.* The centralized ITAP is a particular belief-augmented generalized stochastic online scheduling instance in which:

- jobs are reinterpreted as tasks;
- machines are reinterpreted as agents;
- the revealed task set is generated endogenously through the revelation operator $\mathfrak{R}$;
- state-dependent processing laws are encoded by the productivity maps $\rho_{\tau,o}$;
- multiple agents may redundantly collaborate on a single operation;
- and task completion may follow generalized OR/AND semantics through the families $\mathcal{W}_\tau$.

This is the sense in which the centralized ITAP extends the earlier online stochastic scheduling formalism while remaining within its general conceptual envelope.

Remark (Centralization)*.* The qualifier "centralized" is important. All allocations are chosen by a single planner with a single objective functional Γ. Agents do not have private information, private utilities, or strategic incentives in this model. Those features will be introduced only in the decentralized regime.

3.1.3 Simple Examples

3.2 Approximation Theory

3.3 Complexity Theory

3.4 Implications and Limitations

Chapter 4

VAMP — Formal Definition and Properties

We formalize a **Verifiable Allocation Market POSG (VAMP)** as a market-mediated, partially observable stochastic game that arises when HTAP is decentralized: agents allocate compute across proof/conjecture/publication actions and may additionally trade **verifiable, state-contingent contracts** whose settlement is triggered by a public **Verify** oracle. This section gives a LaTeX-ready VAMP tuple aligned with our existing library/market notation and two-stage transition construction. We then define a contract space (bounties, securities/claims, bundles), admissibility/collateral rules, and a modular market mechanism interface $\mathcal{M}$. We instantiate canonical mechanisms—posted bounties, bilateral OTC contracts, convex-cost/LMSR and CFMM-style AMMs, and combinatorial auctions—and state the appropriate equilibrium concepts (Bayesian Nash, subgame-perfect refinements, Markov perfect equilibria) with existence remarks grounded in fixed-point theorems. Finally, we analyze incentive alignment (bounties internalize publication externalities via a short proof), manipulation modes (front-running, sybil, withholding), core complexity barriers (PPAD/N-EXP), and baseline algorithmic benchmarks that connect directly to the implementation and MARL chapters.

4.1 Mechanistic Impossibility and Decentralization of ITAP

TODO NISAN-RONEN THING

4.2 Partially Observable Stochastic Games

A *partially observable stochastic game* (POSG) is a mathematical model for sequential decision-making with multiple interacting agents under partial information. In this work we focus on the standard *infinite-horizon discounted* formulation.

4.2.1 Definition

An infinite-horizon discounted POSG is specified by the tuple

$$\Big(\mathcal{N}, \mathcal{W}, \{\mathcal{O}^\alpha\}_{\alpha\in\mathcal{N}}, \{\mathcal{Z}^\alpha\}_{\alpha\in\mathcal{N}}, \{\mathcal{A}^\alpha\}_{\alpha\in\mathcal{N}},$$

$$\{\mathrm{Adm}^\alpha\}_{\alpha\in\mathcal{N}}, \mathcal{T}, \{r^\alpha\}_{\alpha\in\mathcal{N}}, \gamma, b_0\Big)$$

where:

- $\mathcal{N}$ is the set of agents, indexed by $\alpha \in \mathcal{N}$.
- $\mathcal{W}$ is the world (state) space.
- $\mathcal{A}^\alpha$ is the (ambient) action space of agent α, and the joint action space is $\mathcal{A} := \prod_{\alpha\in\mathcal{N}} \mathcal{A}^\alpha$.
- $\mathcal{O}^\alpha$ is the observation space of agent α.
- $\mathrm{Adm}^\alpha : \mathcal{W} \times \mathcal{A}^\alpha \to \{0, 1\}$ is the *admissibility predicate* for agent α. It induces the state-dependent legal action set

  $$\mathcal{A}^\alpha(s) := \{a^\alpha \in \mathcal{A}^\alpha : \mathrm{Adm}^\alpha(s, a^\alpha) = 1\}.$$

  We assume $\mathcal{A}^\alpha(s) \neq \varnothing$ for all s (e.g. containing a no-op action).
- $\mathcal{T}$ is the transition kernel: for $s, s' \in \mathcal{W}$ and joint action $a \in \mathcal{A}$,

  $$\mathcal{T}(s' \mid s, a) \in [0, 1]$$

  is the probability of transitioning from s to s' under a.
- $\mathcal{Z}^\alpha$ is the observation kernel for agent α: for $o^\alpha \in \mathcal{O}^\alpha$,

  $$\mathcal{Z}^\alpha(o^\alpha \mid s, a, s') \in [0, 1]$$

  is the probability that agent α observes o^α given transition $s \xrightarrow{a} s'$.
- $r^\alpha : \mathcal{W} \times \mathcal{A} \to \mathbb{R}$ is the per-step reward function for agent α.
- $\gamma \in [0, 1)$ is the discount factor.
- b_0 is the initial distribution over states $s_0 \sim b_0$.

At each timestep t, each agent selects an action $a_t^\alpha \in \mathcal{A}^\alpha(s_t)$, yielding a joint action $a_t = (a_t^\alpha)_{\alpha\in\mathcal{N}} \in \mathcal{A}$. (Equivalently, one may allow agents to propose any $a^\alpha \in \mathcal{A}^\alpha$ and define a convention for inadmissible actions, such as rejection/no-op or penalties; we use explicit admissibility predicates for clarity.)

4.2.2 Policies, Utilities, and Equilibria

A (behavioral) policy for agent α is a mapping from the player's action–observation history to a distribution over actions. Let

$$h_t^\alpha := (o_0^\alpha, a_0^\alpha, o_1^\alpha, a_1^\alpha, \ldots, o_t^\alpha) \in (\mathcal{O}^\alpha \times \mathcal{A}^\alpha)^t \times \mathcal{O}^\alpha$$

denote agent α's history up to time t. A policy $\pi^\alpha(\cdot \mid h_t^\alpha)$ is a distribution over $\mathcal{A}^\alpha$ conditioned on the past history h_t^α. We write Π^α for the space of policies available to agent α.
A *pure strategy* is a map σ^α that selects a single action deterministically as a function of h_t^α.
A joint policy is $\pi := (\pi^\alpha)_{\alpha\in\mathcal{N}}$, and we write $\pi^{-\alpha}$ for the policies of all players except α.
Given a joint policy π, player α's expected discounted return is

$$R^\alpha(\pi) := \mathbb{E}_{\mathcal{T},\mathcal{Z},\pi}\left[\sum_{t=0}^{\infty} \gamma^t\, r^\alpha(s_t, a_t) \;\middle|\; s_0 \sim b_0\right]$$

where the expectation is taken over the the transition kernels T, observation kernels Z, and the (possibly stochastic) action selection induced by π. Under bounded rewards (e.g. $|r^\alpha(s,a)| \le R_{\max}$) and $\gamma \in [0,1)$, this sum converges almost surely and $|R^\alpha(\pi)| \le \frac{R_{\max}}{1-\gamma}$.
Because $R^\alpha(\pi)$ depends on the joint policy and reward functions need not align across players, a POSG is not generally a single-agent optimization problem. Rather, one studies various behavioral policies and game-theoretic equilibria. One such equilibrium is the *Nash equilibrium*: a joint policy π is a Nash equilibrium if every agent α plays a best response to $\pi^{-\alpha}$, i.e., for all policies $\pi^{\alpha'} \in \Pi^\alpha$,

$$R^\alpha(\pi^\alpha, \pi^{-\alpha}) \;\ge\; R^\alpha(\pi^{\alpha'}, \pi^{-\alpha})$$

A particularly important taxonomic feature of POSGs is the relationship between agents' reward functions. In *cooperative* games, all agents share the same reward function: $r^\alpha(s,a) = r^{\alpha'}(s,a)$ for all i, j. In a two-player *zero-sum* game, $r^0(s,a) = -r^1(s,a)$ for all (s,a). When no such restriction is imposed, the POSG is termed *general-sum*.

4.3 VAMP Preliminaries

4.3.1 Syntax

We consider the *formula* ϕ as the core syntactic primitive of (formal) mathematics. Namely, we represent mathematical theorem statements syntactically as a formula (i.e. object of type `Prop`) within some formal language Σ, such as Lean, Rocq, etc. We define $\mathcal{F}$ as the collection of all such formulas, and equip it with an involution $\neg : \mathcal{F} \to \mathcal{F}$; namely, $\neg(\neg\varphi) = \varphi$ for all $\varphi \in \mathcal{F}$. Note that we will refer to formulas as "nodes" interchangably due to a natural graphical structure that will be discussed later.

4.3.2 Semantics

We now aim to represent the semantic meaning of a theorem statement within a particular context concisely. Towards this, we consider the *sequent*, given as an element of $\mathcal{S} := \mathcal{P}_{fin}(\mathcal{F}) \times \mathcal{F}$ and stylized as $[\Gamma \vdash \phi]$ (read: "Gamma *proves* phi"). Namely, such a sequent represents a *proof obligation*, i.e. the notion of proving a theorem statement ϕ (the *consequent*) within the *antecedent* context Γ. Such a proof obligation may be *resolved*, meaning proven.

4.3.3 Libraries

We assume the existence of a universal clock $t \in \mathbb{N}$ and set of agents $\mathcal{N}$, and use it to define a library $\mathcal{L}_\bullet = \{\mathcal{L}_t\}_{t\in\mathbb{N}}$ as a collection of discrete library states that evolves over time. At every discrete timestep $t \in \mathbb{N}$, each library state is given as a tuple

$$\mathcal{L}_t := (\mathcal{C}_t, \mathcal{RS}_t, \delta_t, \Theta_t) \in \mathscr{L}$$

where we partition $\mathcal{F} = \mathcal{C}_t \cup \mathcal{G}_t, \mathcal{C}_t \cap \mathcal{G}_t = \varnothing$ into *concrete* and *ghost* nodes. Intuitively, concrete nodes represent theorem statements that have been stated or conjectured, and ghost nodes represent all hypothetical, possible theorem statements that are yet to be conjectured or stated. $\mathcal{C}_t$ naturally induces a semantic analogue: let $\mathcal{CS}_t := \mathcal{P}_{fin}(\mathcal{C}_t) \times \mathcal{C}_t$ be the set of all *concrete sequents*, which contains a subset $\mathcal{RS}_t \subseteq \mathcal{CS}_t$ of *resolved sequents*, which represent the concrete nodes that have been proven (within a particular antecedent context, of course). Mathematically, a proof of a particular theorem in some context often explicitly depends on other lemmas within this context. In an effort to model this phenomenon, define $\delta_t : \mathcal{RS}_t \to \mathcal{P}_{fin}(\mathcal{F})$ as the dependency mapping. Note that in mathematics, there may exist many proofs of the same theorem with different dependencies for each, and one may find new proofs of the same theorem that are also valid. Thus, in the most general formulation,

we only require that resolved sequents must remain resolved ($\mathcal{RS}_t \subseteq \mathcal{RS}_{t+1}$), though the underlying "proof" (moreso, the set of dependencies of this "proof") can change.
And lastly, we define $\Theta_t : \mathcal{RS}_t \to \mathbb{N}^+ \times \mathcal{N}$ to record the (discrete) solve-time associated to each resolved sequent, and the agent responsible for the resolution.
So, in other words, the library state $\mathcal{L}_t$ represents the collection of all stated theorem statements, the collection of proof witnesses within particular contexts for a subset of these statements, and the dependencies and ancillary metadata of these witnesses.

4.3.3.1 Library Invariants and Properties

Naturally, we impose a monotonicity invariant for libraries: one may not "unconjecture" or "unprove" a theorem. Formally, given a library $\mathcal{L}_\bullet = \{\mathcal{L}_t\}_{t\in\mathbb{N}} = \{(\mathcal{C}_t, \mathcal{RS}_t, \delta_t, \Theta_t)\}_{t\in\mathbb{N}}$, we require that for all $t \in \mathbb{N}$:

$$\mathcal{C}_t \subseteq \mathcal{C}_{t+1} \quad \text{and} \quad \mathcal{RS}_t \subseteq \mathcal{RS}_{t+1}$$

Additionally, we require that if one conjectures a theorem, they must also have automatically conjectured its converse; in other words:

$$\phi \in \mathcal{C}_t \iff \neg\phi \in \mathcal{C}_t$$

Similarly, one may not prove both a theorem and its converse true:

$$[\Gamma \vdash \phi] \in \mathcal{RS}_t \implies [\Gamma \vdash \neg\phi] \notin \mathcal{RS}_t$$

Semantically, we note reflexivity holds: $\phi \in \Gamma \implies [\Gamma \vdash \phi] \in \mathcal{RS}_t$. And for the sake of consistency, we require that for any $[\Gamma \vdash \phi] \in \mathcal{CS}_t$, $\psi \in \Gamma \implies \neg\psi \notin \Gamma$ (although we do not impose any full propositional semantics or richer logical theory behind these abstract formulas, preferring to leave them fully symbolic objects with various resolution and concreteness states, allowing inconsistent antecedents in our witnesses limits the scalability of our approach to deployment as a strategy module for multiagent theorem proving in a fully-defined formal language and type theory).
Note that this last condition implicitly redefines

$$\mathcal{CS}_t := \mathcal{P}_{fin}(\mathcal{C}_t)/\sim \times \mathcal{C}_t$$

where $\phi \sim \neg\phi$ is the equivalence relation generated by the negation operation. So, due to this consistency condition, we note that $\mathcal{RS}_t$ is upwards-closed, in that if $[\Gamma \vdash \phi] \in \mathcal{RS}_t$, then for all $\Gamma' \in \mathcal{P}_{fin}(\mathcal{C}_t)$, $\Gamma \subseteq \Gamma' \implies [\Gamma' \vdash \phi] \in \mathcal{RS}_t$.
Additionally, considering the dependency mapping, we note that intuitively δ_t must satisfy the property that $\delta_t([\Gamma \vdash \phi]) \subseteq \Gamma$. Moreover, one may use δ_t to induce a directed graph

$D(\delta_t) := (\mathcal{CS}_t, \mathcal{E}_t)$, where

$$\mathcal{E}_t = \Big\{([\Gamma \vdash \phi], [\Gamma' \vdash \psi]) : \phi \in \delta_t([\Gamma' \vdash \psi]), \Gamma \subseteq \Gamma'\Big\}$$

We require that this induced directed graph $D(\delta_t)$ of δ_t must be acyclic.

4.3.3.2 Axioms and Fully Resolved Sequents

As an aside, we additionally note that although $[\Gamma \vdash \phi] \in \mathcal{RS}_t$ may be marked resolved, that does not mean that ϕ has a "error"-free proof within the current library state as its proof may depend on some still-unresolved lemmas. More concretely, we may define $\mathcal{FRS}_t \subseteq \mathcal{RS}_t$ as the collection of *fully resolved sequents*, which is defined inductively such that *axioms* $[\vdash \phi] \in \mathcal{FRS}_t$ and $[\Gamma \vdash \phi] \in \mathcal{FRS}_t$ if and only if for all $\phi' \in \delta_t([\Gamma \vdash \phi])$, there exists $\Gamma' \subseteq \Gamma$ such that $[\Gamma' \vdash \phi'] \in \mathcal{FRS}_t$.

4.3.3.3 Private and Shared Libraries

In the multiagent setting, each agent $\alpha \in \mathcal{N}$ maintains a *private library* $\mathcal{L}^\alpha_\bullet = \{\mathcal{L}^\alpha_t\}_{t\in\mathbb{N}}$ with

$$\mathcal{L}^\alpha_t := (\mathcal{C}^\alpha_t, \mathcal{RS}^\alpha_t, \delta^\alpha_t, \Theta^\alpha_t),$$

and the environment maintains a *shared (public) library* $\mathcal{L}^0_\bullet = \{\mathcal{L}^0_t\}_{t\in\mathbb{N}}$ with

$$\mathcal{L}^0_t := (\mathcal{C}^0_t, \mathcal{RS}^0_t, \delta^0_t, \Theta^0_t).$$

We additionally enforce the following inclusion invariant for all $t \in \mathbb{N}, \alpha \in \mathcal{N}$:

$$\mathcal{L}^0_t \subseteq \mathcal{L}^\alpha_t$$

meaning $\mathcal{C}^0_t \subseteq \mathcal{C}^\alpha_t$, $\mathcal{RS}^0_t \subseteq \mathcal{RS}^\alpha_t$, and for all $s \in \mathcal{RS}^0_t, \delta^0_t(s) = \delta^\alpha_t(s)$ and $\Theta^0_t(s) = \Theta^\alpha_t(s)$.

4.3.4 Market State

We additionally model the market as a discrete-time process $\mathcal{M}_\bullet := \{\mathcal{M}_t\}_{t\in\mathbb{N}}$. At each timestep t, the market state is

$$\mathcal{M}_t := \big(\{\mathcal{P}^\alpha_t\}_{\alpha\in\mathcal{N}},\ \mathcal{ML}_t\big) \in \mathfrak{P}^n \times \mathfrak{L} = \mathfrak{M}$$

where $\mathcal{P}^\alpha_t \in \mathfrak{P}$ is agent α's private portfolio state and $\mathcal{ML}_t \in \mathfrak{L}$ is the public market ledger. We assume a canonical balance/valuation map $\beta : \mathfrak{P} \to \mathbb{R}$ and define the balance of agent α at time t by

$$B^\alpha_t := \beta(\mathcal{P}^\alpha_t) \in \mathbb{R}.$$

4.3.5 Action Primitives

We now specify the core action primitives of VAMP. These primitives will later be assembled into the POSG transition and observation structure; for now, we define them as standalone kernels and update maps.

4.3.5.1 Discrete kernels.

All stochastic primitives in this section have finite or countable output spaces, and may be treated as probability mass functions (pmfs). Concretely, for an input space X and discrete output space Y, a stochastic kernel is a map $K : Y \times X \to [0, 1]$ such that $\sum_{y \in Y} K(y \mid x) = 1$ for all $x \in X$.

4.3.5.2 Proof Kernel

Fix an agent $\alpha \in \mathcal{N}$. The proof kernel models allocating a time budget to attempt to resolve a target sequent.
Let the input space be

$$X_{\text{proof}} := \mathscr{L} \times \mathcal{S} \times \mathbb{N}^+,$$

where (s, τ) consists of a target sequent $s = [\Gamma \vdash \phi]$ and a discrete time budget τ. Let the output space be

$$Y_{\text{proof}} := \{(0, \perp)\} \ \cup \ \{(1, d) : d \in \mathcal{P}_{fin}(\mathcal{F})\},$$

where $(0, \perp)$ denotes failure, and $(1, d)$ denotes success together with a dependency set d.
The proof kernel for agent α is a pmf

$$\mathcal{K}^{\alpha}_{\text{proof}} : Y_{\text{proof}} \times X_{\text{proof}} \to [0, 1]$$

Although $\mathcal{K}^{\alpha}_{\text{proof}}$ is defined on all of $\mathcal{S} \times \mathbb{N}^+$, in the intended game dynamics the agent is only allowed to invoke a proof attempt on *concrete* targets available in its local library (later captured via a state-dependent admissibility predicate). On admissible inputs $(\mathcal{L}, [\Gamma \vdash \phi], \tau)$, a successful output $(1, d)$ is intended to satisfy $d \subseteq \Gamma$.
Upon success, the environment will (i) mark the target sequent resolved in α's library and (ii) record/update its dependency map at that sequent with the returned d, subject to global consistency (e.g., preserving acyclicity of the induced dependency graph). Upon failure, no new resolved sequent is added. These effects are enforced later by the global state-update operator.

4.3.5.3 Conjecture Kernel

Fix an agent $\alpha \in \mathcal{N}$. The conjecture kernel models allocating a time budget to propose a new formula, anchored at an existing sequent.
Let the input space be

$$X_{\text{conj}} := \mathscr{L} \times \mathcal{S} \times \mathbb{N}^+,$$

where (s, τ) consists of an anchor sequent $s = [\Gamma \vdash \phi]$ and time budget τ. Let the output space be

$$Y_{\text{conj}} := \{(0, \bot)\} \ \cup \ \{(1, \psi) : \psi \in \mathcal{F}\},$$

where $(0, \bot)$ denotes failure and $(1, \psi)$ denotes successful conjecture of formula ψ.
The conjecture kernel for agent α is a pmf

$$\mathcal{K}^{\alpha}_{\text{conj}} : Y_{\text{conj}} \times X_{\text{conj}} \to [0, 1].$$

As with the proof kernel, $\mathcal{K}^{\alpha}_{\text{conj}}$ is only intended to be invoked on anchors available to the agent (e.g., concrete sequents in α's local library).
Moreover, given admissible input $(\mathcal{L}, [\Gamma \vdash \phi], \tau)$ and a successful output $(1, \psi) \sim \mathcal{K}^{\alpha}_{\text{conj}}(\cdot \mid \mathcal{L}, [\Gamma \vdash \phi], \tau)$, we enforce that ψ is a ghost node. Namely, if $\mathcal{L} = (\mathcal{C}, \mathcal{RS}, \delta, \Theta)$, then $\psi \in \mathcal{F} \backslash \mathcal{C}$. Upon such success with output $(1, \psi)$, we intend the environment to add ψ to α's set of concrete formulas (and also add $\neg\psi$ to preserve the negation-pair invariant). This converts ψ from ghost to concrete relative to α.

4.3.5.4 Dependency Closure for Publication

Publication is a deterministic primitive that copies a closure-completed subset of an agent's private library into the shared library, and (by the shared-library invariant) into every agent's local library.
Fix agent α and time t, and suppose δ^{α}_t induces the directed acyclic graph $D(\delta^{\alpha}_t) = (\mathcal{CS}^{\alpha}_t, \mathcal{E}^{\alpha}_t)$ as defined earlier. For any set of target resolved sequents $R \subseteq \mathcal{RS}^{\alpha}_t$, define its *dependency closure*:

$$\begin{aligned} \text{cl}^{\alpha}_t(R) := \Big\{ & s \in \mathcal{RS}^{\alpha}_t : \exists r \in R \text{ such that there} \\ & \text{is a directed path } s \to r \text{ in } D(\delta^{\alpha}_t) \Big\} \end{aligned}$$

Equivalently, $\text{cl}^{\alpha}_t(R)$ is the set of all resolved sequents required (transitively) as dependencies of the sequents in R.

We now proceed to extend this closure operator to the payload for such publication actions. Namely, a publication payload is a pair $U_t^\alpha = (C, R)$ with $C \subseteq \mathcal{C}_t^\alpha, \;\; R \subseteq \mathcal{RS}_t^\alpha$.
Define its closure-completion $\overline{U_t^\alpha} := \big(\overline{C}, \overline{R}\big)$ where $\overline{R} := \mathrm{cl}_t^\alpha(R)$ and

$$\overline{C} := C \;\cup \bigcup_{[\Gamma \vdash \phi] \in \overline{R}} (\Gamma \cup \{\phi\})$$

Thus, publishing resolved sequents forces publication of all formulas appearing in the antecedents and consequents of the dependency-closed witness set.
Therefore, given $\overline{U_t^\alpha}$, intuitively, the environment updates the shared library by unioning in $\overline{C}$ and $\overline{R}$ (and any associated dependency information it chooses to expose), then propagates these additions to every agent's local library so that $\mathcal{L}_{t+1}^0 \subseteq \mathcal{L}_{t+1}^\beta$ continues to hold for all β.

4.3.5.5 Query Model

Each agent α maintains a stateful *query model* which estimates how its prover stack will perform on a requested proof search. Intuitively, the query model is a continually-updated predictor for (i) the probability of successfully resolving a target sequent within a given time budget and (ii) the expected time-to-solution (or a proxy thereof).
Fix an abstract space $\mathcal{Q}$ of query models (e.g., parameters of a Bayesian estimator, weights of a learned predictor, or a table of sufficient statistics). At each timestep t, agent α has a current query model $Q_t^\alpha \in \mathcal{Q}$. A query operation takes as input a target (concrete) sequent:

$$q^\alpha : \mathcal{Q} \times \mathcal{S} \to [0,1] \times \mathbb{N}$$

and write

$$(\hat{p}_t^\alpha(s), \hat{\tau}_t^\alpha(s)) := q^\alpha(Q_t^\alpha, s),$$

where $\hat{p}_t^\alpha([\Gamma \vdash \phi])$ is the model's estimate of $\Pr(\phi \text{ is true})$ and $\hat{\tau}_t^\alpha(s)$ is an estimate of the expected budget required to successfully prove $[\Gamma \vdash \phi]$.
More explicitly, for a fixed agent α and library state $\mathcal{L}$, and target sequent $s \in \mathcal{S}$, define the probability of success within budget τ by

$$p_\tau^\alpha(s) \;:=\; \sum_{d \in \mathcal{P}_{fin}(\mathcal{F})} \mathcal{K}_{\text{proof}}^\alpha\big((1,d) \mid \mathcal{L}, s, \tau\big).$$

This induces a time-to-solution distribution by declaring a random variable T_s^α with CDF

$$\Pr\big(T_s^\alpha \le \tau\big) \;:=\; p_\tau^\alpha(s), \qquad \tau \in \mathbb{N}^+.$$

We interpret the eventual solvability (or "truth/resolvability" under α's prover stack) as

$$p_\infty^\alpha(s) \;:=\; \sup_{\tau\in\mathbb{N}^+} p_\tau^\alpha(s),$$

and, when $p_\infty^\alpha(s) > 0$, the conditional expected solve time as

$$\tau_\infty^\alpha(s) \;:=\; \mathbb{E}[T_s^\alpha \,|\, T_s^\alpha < \infty] = \sum_{k\geq 1} \Pr(T_s^\alpha \geq k \mid T_s^\alpha < \infty)$$

Accordingly, the query readout $q^\alpha(Q_t^\alpha, s) = (\hat{p}_t^\alpha(s), \hat{\tau}_t^\alpha(s))$ is intended to approximate

$$\hat{p}_t^\alpha(s) \approx p_\infty^\alpha(s), \qquad \hat{\tau}_t^\alpha(s) \approx \tau_\infty^\alpha(s),$$

using the agent's accumulated evidence.

The query model is updated from observed outcomes of proof attempts (and optionally from public disclosures).

Define the space of query-training data as

$$\mathcal{D}(\mathcal{Q}) := \mathcal{S} \times (\mathbb{N}^+ \times \mathcal{N}),$$

where a datum $(s, (\tau, \alpha'))$ denotes a successful proof attempt by agent α' on s on time budget τ.

Define an update operator

$$\Delta^\alpha : \mathcal{Q} \times \mathcal{D}(\mathcal{Q}) \to \mathcal{Q}$$

Given an observed training datum $d_t^\alpha \in \mathcal{D}(\mathcal{Q})$ available to agent α at time t, the query model updates as

$$Q_{t+1}^\alpha := \Delta^\alpha(Q_t^\alpha, d_t^\alpha),$$

We enforce the commutativity and associativity of Δ^α, so that if multiple data points $\{(d_t^\alpha)_i\}_{i=0}^{k-1}$ are observed in a single timestep, there is a single and well-defined batch update which is the composition (in any order) of the individual datum updates: i.e., if $Q_t^\alpha = (Q_t^\alpha)_0$,

$$(Q_t^\alpha)_{i+1} = \Delta^\alpha((Q_t^\alpha)_i, (d_t^\alpha)_i)$$

$$Q_{t+1}^\alpha := (Q_t^\alpha)_{k-1}$$

which is consistent across all possible permutations on $\{0, \cdots, k-1\}$.

The framework does not prescribe the internal form of Q_t^α, and allows downstream experiments to arbitrarily instantiate $(\mathcal{Q}, q^\alpha, \Delta^\alpha)$ as desired, whether that means querying based on Bayesian update rules, learned predictors, or any other mechanism.

4.3.5.6 Market Mechanism

We briefly take an aside to define the *market mechanism* as a modular plugin that that governs (i) which market actions are available, (ii) which are admissible at a given world state, and (iii) how the market state evolves. One may parameterize and study many different verifiable allocation games by simply switching out this underlying economic reward mechanism.

We define a market mechanism $\mathscr{M}$ as a tuple:

$$\mathscr{M} := \Big(\mathcal{N}, \mathfrak{M}, \beta, \mathcal{W}, \{\mathcal{A}^{\alpha}_{\text{mkt}}\}_{\alpha\in\mathcal{N}}, \\ \{\text{Adm}^{\alpha}_{\text{mkt}}\}_{\alpha\in\mathcal{N}}, \mathcal{T}_{\text{mkt}}\Big)$$

where:

- $\mathcal{N}$ is the set of agents, indexed by $\alpha \in \mathcal{N}$.
- $\mathfrak{M} = (\mathfrak{P}^n, \mathfrak{L})$ is the market state space, where $\mathfrak{P}$ is the space of possible portfolios and $\mathfrak{L}$ is the space of possible public market ledgers.
- $\beta : \mathfrak{P} \to \mathbb{R}$ is the canonical balance/valuation map.
- $\mathcal{W}$ is the world (state) space. Note that $\mathfrak{M}$ is a component in $\mathcal{W}$.
- $\mathcal{A}^{\alpha}_{\text{mkt}}$ is the (ambient) *market* action space of agent α. Note that the market no-op is always a valid action: $\perp_{\text{mkt}} \in \mathcal{A}^{\alpha}_{\text{mkt}}$. The joint market action space is:

$$\mathcal{A}_{\text{mkt}} := \prod_{\alpha\in\mathcal{N}} \mathcal{A}^{\alpha}_{\text{mkt}}$$

- $\text{Adm}^{\alpha}_{\text{mkt}} : \mathcal{W} \times \mathcal{A}^{\alpha}_{\text{mkt}} \to \{0, 1\}$ is the market admissibility predicate for agent α. It induces the state-dependent admissible market action set:

$$\mathcal{A}^{\alpha}_{\text{mkt}}(s) := \{a^{\alpha} \in \mathcal{A}^{\alpha}_{\text{mkt}} : \text{Adm}^{\alpha}_{\text{mkt}}(s, a^{\alpha}) = 1\}.$$

- $\mathcal{T}_{\text{mkt}} : \Big(\mathcal{W} \times \mathcal{W} \times \mathcal{A}_{\text{mkt}}\Big) \times \Sigma_{\mathcal{W}} \to [0, 1]$ is a stochastic transition kernel with input space $\mathcal{W} \times \mathcal{W} \times \mathcal{A}_{\text{mkt}}$ and output space $\mathcal{W}$. Namely, given a change in world state from $w \to w' \in \mathcal{W}$, and a joint market action $a_{\text{mkt}} \in \mathcal{A}_{\text{mkt}}$, this kernel induces a distribution over the next world state:

$$\bar{w} \sim \mathcal{T}_{\text{mkt}}(\cdot \mid w, w', a_{\text{mkt}}).$$

  Deterministic mechanisms are recovered as Dirac kernels.

4.4 Verifiable Allocation Market POSG

We now define the *Verifiable Allocation Market POSG* (VAMP) as a particular POSG designed to model multiagent verifiable subtask allocation mediated by an arbitrary market mechanism – interpreted as an abstracted decentralized formal theorem proving environment.

Fix a set of agents $\mathcal{N}$. A VAMP is a class of infinite-horizon discounted POSG's, parameterized by a particular market mechanism $\mathscr{M}$:

$$\Big(\mathcal{N}, \mathcal{W}, \{\mathcal{O}^\alpha\}_{\alpha\in\mathcal{N}}, \{\mathcal{Z}^\alpha\}_{\alpha\in\mathcal{N}}, \{\mathcal{A}^\alpha\}_{\alpha\in\mathcal{N}},$$

$$\{\mathrm{Adm}^\alpha\}_{\alpha\in\mathcal{N}}, \mathcal{T}, \{r^\alpha\}_{\alpha\in\mathcal{N}}, \gamma, b_0, \mathscr{M}\Big)$$

defined as follows.

4.4.1 World State Space $\mathcal{W}$

A world state at time t is a tuple

$$w_t := \Big(\{\mathcal{L}_t^\alpha\}_{\alpha\in\mathcal{N}},\ \mathcal{L}_t^0,\ \mathcal{M}_t,\ \{J_t^\alpha\}_{\alpha\in\mathcal{N}},\ \{Q_t^\alpha\}_{\alpha\in\mathcal{N}},$$
$$\{\Omega_t^\alpha\}_{\alpha\in\mathcal{N}},\ \{U_{t,\mathrm{proof}}^\alpha\}_{\alpha\in\mathcal{N}},\ \{U_{t,\mathrm{conj}}^\alpha\}_{\alpha\in\mathcal{N}}\Big) \in \mathcal{W},$$

where:

- $\mathcal{L}_t^\alpha$ is agent α's private library and $\mathcal{L}_t^0$ is the shared public library.
- $\mathcal{M}_t = (\{\mathcal{P}_t^\alpha\}_{\alpha\in\mathcal{N}},\ \mathcal{ML}_t) \in \mathfrak{P}^n \times \mathfrak{L} = \mathfrak{M}$ is the market state, as defined in $\mathscr{M}$.
- J_t^α is the durative job descriptor

  $$J_t^\alpha \in \{\bot\} \ \cup\ \big(\{\mathrm{Proof}, \mathrm{Conj}\} \times \mathcal{S} \times \mathbb{N}^+ \times \mathbb{N}^+\big)$$

  with $\bot$ meaning idle and $(\cdot, s, \tau_{\mathrm{rem}}, \tau_{\mathrm{eff}})$ meaning: (i) τ_{rem} steps remain in the *current run* and (ii) τ_{eff} is the *effective cumulative budget* used to evaluate the run upon completion. Concretely, when a run is initiated at time t with intended additional duration τ_{dur} on target s, we set $\tau_{\mathrm{eff}} := U_{t,-}^\alpha(s) + \tau_{\mathrm{dur}}$ and $\tau_{\mathrm{rem}} := \tau_{\mathrm{dur}}$. The agent is forced to take τ_{dur} consecutive no-op actions, and at completion the outcome is sampled from the appropriate kernel using budget τ_{eff}.
- $Q_t^\alpha \in \mathcal{Q}$ is agent α's query model state, updated by Δ^α on successful proofs.

- $\Omega_t^\alpha \in \{\perp\} \sqcup (\mathcal{S} \times [0,1] \times \mathbb{N})$ is agent α's last query response, either empty $\perp$ or a triple $(s, \hat{p}, \hat{\tau})$ signifying an estimate that the sequent s is true with probability $\hat{p}$, and given it is true, an estimate of the time it would take to prove it $\hat{\tau}$.

- $U_{t,-}^\alpha : \mathcal{S} \to \mathbb{N}$ records the total time already spent by agent $\alpha \in \mathcal{N}$ attempting a proof or conjecture for sequent $s \in \mathcal{S}$, up to time t. This is used to allow agents to "pick up where they left off". Note $U_{0,-}^\alpha \equiv 0$.

We take $\mathcal{W}$ to be the set of tuples satisfying all such tuples satisfying the consitituent objects' various invariants and conditions.

4.4.2 Observations $\mathcal{O}^\alpha$ and Observation Kernels $\mathcal{Z}^\alpha$

Agent α observes

$$o_t^\alpha := \big(\mathcal{L}_t^\alpha, \mathcal{P}_t^\alpha, B_t^\alpha, \Omega_t^\alpha;\ \mathcal{L}_t^0, \mathcal{ML}_t\big) \in \mathcal{O}^\alpha.$$

The observation kernel is deterministic:

$$\mathcal{Z}^\alpha(o^\alpha \mid w, a, w') = \mathbf{1}[\, o^\alpha = \mathrm{Obs}^\alpha(w)\,],$$

where $\mathrm{Obs}^\alpha(w)$ computes the observation slice as above.

4.4.3 Action Space $\mathcal{A}^\alpha$ and Admissibility Adm^α

For each agent α, define the tagged *non-market* action space

$$\begin{aligned}\mathcal{A}_{\mathrm{nm}}^\alpha :=& \{\mathrm{Prove}\} \times (\mathcal{S} \times \mathbb{N}^+) \\ &\sqcup \{\mathrm{Conj}\} \times (\mathcal{S} \times \mathbb{N}^+) \\ &\sqcup \{\mathrm{Pub}\} \times (\mathcal{P}_{fin}(\mathcal{F}) \times \mathcal{P}_{fin}(\mathcal{S})) \\ &\sqcup \{\mathrm{Qry}\} \times \mathcal{S} \\ &\sqcup \{\perp_{\mathrm{nm}}\}\end{aligned}$$

Where $\perp_{\mathrm{nm}}$ indicates a non-market no-op. Thus, the full action space for agent α may be written as:

$$\mathcal{A}^\alpha = \mathcal{A}_{\mathrm{nm}}^\alpha \sqcup \mathcal{A}_{\mathrm{mkt}}^\alpha$$

where $\mathcal{A}_{\mathrm{mkt}}^\alpha$ is specified in $\mathscr{M}$. Note then that as an agent α may only take one action at a given timestep, both $\perp_{\mathrm{nm}}$ and $\perp_{\mathrm{mkt}}$ would indicate a no-op.
As such, the joint action space is

$$\mathcal{A} := \prod_{\alpha \in \mathcal{N}} \mathcal{A}^\alpha$$

Note that for a given joint action $a \in \mathcal{A}$, we may uniquely decompose it into a non-market and market joint action. In other words:

$$a = a_{\text{nm}} \oplus a_{\text{mkt}}$$

Where for each agent-component of the joint action $a^\alpha \in \mathcal{A}^\alpha$, if $a^\alpha \in \mathcal{A}^\alpha_{\text{nm}}$, then let $a^\alpha_{\text{nm}} = a^\alpha$ and $a^\alpha_{\text{mkt}} = \perp_{\text{mkt}}$. Alternatively, if $a^\alpha \in \mathcal{A}^\alpha_{\text{mkt}}$, then let $a^\alpha_{\text{mkt}} = a^\alpha$ and $a^\alpha_{\text{nm}} = \perp_{\text{nm}}$. Thus, we may define:

$$a_{\text{nm}} = \prod_{\alpha \in \mathcal{N}} a^\alpha_{\text{nm}}, \qquad a_{\text{mkt}} = \prod_{\alpha \in \mathcal{N}} a^\alpha_{\text{mkt}}$$

Now, define $\text{Adm}^\alpha : \mathcal{W} \times \mathcal{A}^\alpha \to \{0, 1\}$ as an admissibility predicate, which enforces semantic and operational constraints.
Fix $w_t \in \mathcal{W}$ and consider $\alpha \in \mathcal{N}$. Then:

- If $J^\alpha_t = \perp$, then:
  - $\text{Adm}^\alpha(w_t, \perp) = 1$.
  - For $a^\alpha_t = (\text{Prove}, (s, \tau))$ and $a^\alpha_t = (\text{Conj}, (s, \tau))$:

    $$\text{Adm}^\alpha(w_t, a^\alpha_t) = 1 \iff s \in \mathcal{CS}^\alpha_t$$

  - For $a^\alpha_t = (\text{Pub}, (C, R))$:

    $$\text{Adm}^\alpha(w_t, a^\alpha_t) = 1 \iff C \subseteq \mathcal{C}^\alpha_t,\ R \subseteq \mathcal{RS}^\alpha_t$$

  - For $a^\alpha_t = (\text{Qry}, s)$:

    $$\text{Adm}^\alpha(w_t, a^\alpha_t) = 1 \iff s \in \mathcal{CS}^\alpha_t$$

  - For market actions $a^\alpha_t \in \mathcal{A}^\alpha_{\text{mkt}}$:

    $$\text{Adm}^\alpha(w_t, a^\alpha_t) = \text{Adm}^\alpha_{\text{mkt}}(w_t, a^\alpha_t)$$

    where $\text{Adm}^\alpha_{\text{mkt}}$ is specified by $\mathscr{M}$.

- If $J^\alpha \neq \perp$, then the agent is forced to take no-op: $\text{Adm}^\alpha(w_t, \perp) = 1$ and for all $a^\alpha_t \in \mathcal{A}^\alpha \setminus \{\perp\}$, $\text{Adm}^\alpha(w_t, a^\alpha_t) = 0$.

4.4.4 Transition Kernel $\mathcal{T}$

We define $\mathcal{T}(\cdot \mid w_t, a_t)$ by a two-stage generative construction. Namely, given an admissible input pair $(w_t, a_t) \in \mathcal{W} \times \mathcal{A}$, uniquely decompose the joint action as

$$a_t = a_{t,\mathrm{nm}} \oplus a_{t,\mathrm{mkt}}$$

We then define a non-market transisition kernel $\mathcal{T}_{\mathrm{nm}} : (\mathcal{W} \times \mathcal{A}_{\mathrm{nm}}) \times \Sigma_{\mathcal{W}} \to [0,1]$ and consider the market transition kernel $\mathcal{T}_{\mathrm{mkt}} : (\mathcal{W} \times \mathcal{W} \times \mathcal{A}_{\mathrm{mkt}}) \times \Sigma_{\mathcal{W}} \to [0,1]$ as specified by $\mathscr{M}$. Intuitively, the full transition kernel $\mathcal{T}$ is the composition of the two stages: first draw $w'_t \sim \mathcal{T}_{\mathrm{nm}}(\cdot \mid w_t, a_{t,\mathrm{nm}})$, then draw $w_{t+1} \sim \mathcal{T}_{\mathrm{mkt}}(\cdot \mid w_t, w'_t, a_{t,\mathrm{mkt}})$.
We now proceed to define each stage of this construction.

4.4.4.1 Background: Deterministic Pre-Update

We introduce a deterministic map that serves as the first step of the non-market transition kernel. It applies everything that happens immediately from the joint non-market action at this step, before any stochastic proof/conjecture outcomes are realized. So, given a world state $w \in \mathcal{W}$, we define:

$$\tilde{w} := \mathrm{Pre}(w, \mathbf{a}_{\mathrm{nm}})$$

as the world state obtained by applying the following deterministic edits in order:

1. Decrement job timers. Namely, for each $\alpha \in \mathcal{N}$:
   - If $J^\alpha = \perp$, leave it.
   - If $J^\alpha = (\mathrm{Proof}, s, \tau_{\mathrm{rem}}, \tau_{\mathrm{tot}})$ or $(\mathrm{Conj}, s, \tau_{\mathrm{rem}}, \tau_{\mathrm{tot}})$ with $\tau_{\mathrm{rem}} > 0$, set $\tau_{\mathrm{rem}} \leftarrow \tau_{\mathrm{rem}} - 1$.
2. Start new proof/conjecture jobs if idle. Namely, for each $\alpha \in \mathcal{N}$ with $J^\alpha = \perp$:
   - If $a^\alpha_{\mathrm{nm}} = (\mathrm{Prove}, (s, \tau))$, set $J^\alpha \leftarrow (\mathrm{Proof}, s, \tau, \tau + U^\alpha_{\mathrm{proof}}(s))$.
   - If $a^\alpha_{\mathrm{nm}} = (\mathrm{Conj}, (s, \tau))$, set $J^\alpha \leftarrow (\mathrm{Conj}, s, \tau, \tau + U^\alpha_{\mathrm{conj}}(s))$.
   - Otherwise do nothing.
3. Apply publication actions and propagate to shared library. Namely, for each $\alpha \in \mathcal{N}$, if $a^\alpha_{\mathrm{nm}} = (\mathrm{Pub}, (C^\alpha, R^\alpha))$, then consider the payload $U^\alpha := (C^\alpha, R^\alpha)$. Compute the closure $\overline{U^\alpha}$ and update the shared library and query model. Namely, let $\mathcal{C}^0_{\mathrm{old}} := \mathcal{C}^0$ and $\mathcal{RS}^0_{\mathrm{old}} := \mathcal{RS}^0$, and update:

$$\mathcal{C}^0 \leftarrow \mathcal{C}^0 \cup \bigcup_\alpha \overline{C}^\alpha, \qquad \mathcal{RS}^0 \leftarrow \mathcal{RS}^0 \cup \bigcup_\alpha \overline{R}^\alpha$$

Then for all $s \in \mathcal{RS}^0 \setminus \mathcal{RS}^0_{\text{old}}$:

$$\delta^0(s) \leftarrow \delta^\alpha(s), \qquad \Theta^0(s) \leftarrow \Theta^\alpha(s)$$

Then propagate public additions to each private library to maintain the invariant $\mathcal{L}^0 \subseteq \mathcal{L}^\beta$:

$$\mathcal{C}^\beta \leftarrow \mathcal{C}^\beta \cup (\mathcal{C}^0 \setminus \mathcal{C}^0_{\text{old}})$$

$$\mathcal{RS}^\beta \leftarrow \mathcal{RS}^\beta \cup (\mathcal{RS}^0 \setminus \mathcal{RS}^0_{\text{old}})$$

Also update for all $s \in \mathcal{RS}^0 \setminus \mathcal{RS}^0_{\text{old}}$:

$$\delta^\beta(s) \leftarrow \delta^0(s), \qquad \Theta^\beta(s) \leftarrow \Theta^0(s)$$

$$Q^\beta \leftarrow \Delta^\beta(Q^\beta, (s, \Theta^0(s)))$$

4. Update query buffer. Namely, for each $\alpha \in \mathcal{N}$, if $a^\alpha_{\text{nm}} = (\text{Qry}, s)$, set

$$\Omega^\alpha \leftarrow \big(s, \hat{p}, \max\{0, \hat{\tau} - U^\alpha_{\text{proof}}(s)\}\big)$$

where $(\hat{p}, \hat{\tau}) := q^\alpha(Q^\alpha, s)$. Otherwise set $\Omega^\alpha \leftarrow \perp$.

4.4.4.2 Background - Stochastic Job Completion

Given $\tilde{w}$, define completion index sets:

$$\mathsf{Comp}_{\text{proof}}(\tilde{w}) = \{\alpha : J^\alpha(\tilde{w}) = (\text{Proof}, s, 0, \tau) \text{ for some } s, \tau\}$$

$$\mathsf{Comp}_{\text{conj}}(\tilde{w}) = \{\alpha : J^\alpha(\tilde{w}) = (\text{Conj}, s, 0, \tau) \text{ for some } s, \tau\}$$

For each $\alpha \in \mathsf{Comp}_{\text{proof}}(\tilde{w})$, let $(s^\alpha_{\text{proof}}(\tilde{w}), \tau^\alpha_{\text{proof}}(\tilde{w}))$ be the (s, τ) stored in $J^\alpha(\tilde{w})$ (and similarly for conjecture completions).
Then, we may now define outcome spaces (per completing job):

$$Y^\alpha_{\text{proof}} := \{(0, \perp)\} \cup \{(1, d) : d \in \mathcal{P}_{\text{fin}}(\mathcal{F})\}$$

$$Y^\alpha_{\text{conj}} := \{(0, \perp)\} \cup \{(1, \psi) : \psi \in \mathcal{F}\}$$

And the joint outcome space, dependent on $\tilde{w}$:

$$Y(\tilde{w}) := \prod_{\alpha \in \mathsf{Comp}_{\text{proof}}(\tilde{w})} Y^{\alpha}_{\text{proof}} \times \prod_{\alpha \in \mathsf{Comp}_{\text{conj}}(\tilde{w})} Y^{\alpha}_{\text{conj}}$$

Thus, we may define a product measure over this joint outcome space by conditional independence across completing jobs:

$$\mathbb{K}_{\tilde{w}}(dy) := \left(\prod_{\alpha \in \mathsf{Comp}_{\text{proof}}(\tilde{w})} \mathcal{K}^{\alpha}_{\text{proof}}(dy^{\alpha} \mid \mathcal{L}^{\alpha}(\tilde{w}), s^{\alpha}_{\text{proof}}(\tilde{w}), \tau^{\alpha}_{\text{proof}}(\tilde{w})) \right) \cdot \left(\prod_{\alpha \in \mathsf{Comp}_{\text{conj}}(\tilde{w})} \mathcal{K}^{\alpha}_{\text{conj}}(dy^{\alpha} \mid \mathcal{L}^{\alpha}(\tilde{w}), s^{\alpha}_{\text{conj}}(\tilde{w}), \tau^{\alpha}_{\text{conj}}(\tilde{w})) \right)$$

4.4.4.3 Background - Deterministic Post-Update

We now aim to deterministically apply the realized outcomes $y \in Y(\tilde{w})$ to update libraries, dependency maps, solve-times, query models, and clear completed jobs.
Given $\tilde{w}$ and $y \in Y(\tilde{w})$, define

$$w^{+} := \text{Post}_{\text{nm}}(\tilde{w}, y)$$

by applying:

1. Resolve completing proof jobs. Namely, for each $\alpha \in \mathsf{Comp}_{\text{proof}}(\tilde{w})$, let

   $$y^{\alpha} = (b^{\alpha}, z^{\alpha}) \in Y^{\alpha}_{\text{proof}}$$

   Let $(s^{\alpha}, \tau^{\alpha})$ be the job's stored target and total budget. Regardless of proof success, set $U^{\alpha}_{\text{proof}}(s^{\alpha}) \leftarrow \tau^{\alpha}$.

   Then, if $b^{\alpha} = 1$ and $z^{\alpha} = d$, then update:

   - $\mathcal{RS}^{\alpha} \leftarrow \mathcal{RS}^{\alpha} \cup \{s^{\alpha}\}$
   - $\delta^{\alpha}(s^{\alpha}) \leftarrow d$ (after enforcing $d \subseteq \Gamma$ / acyclicity checks)
   - $\Theta^{\alpha}(s^{\alpha}) \leftarrow (\tau^{\alpha}, \alpha)$
   - $Q^{\alpha} \leftarrow \Delta^{\alpha}(Q^{\alpha}, (s^{\alpha}, \Theta^{\alpha}(s^{\alpha})))$

2. Resolve completing conjecture jobs. Namely, for each $\alpha \in \mathsf{Comp}_{\text{conj}}(\tilde{w})$, let

   $$y^{\alpha} = (b^{\alpha}, z^{\alpha}) \in Y^{\alpha}_{\text{conj}}$$

Regardless of conjecture success, set $U^{\alpha}_{\text{conj}}(s^{\alpha}) \leftarrow \tau^{\alpha}$. Then, if $b^{\alpha} = 1$ and $z^{\alpha} = \psi$, then:

$$\mathcal{C}^{\alpha} \leftarrow \mathcal{C}^{\alpha} \cup \{\psi, \neg\psi\}$$

3. Clear completed jobs. Namely, for each $\alpha \in \mathsf{Comp}_{\text{proof}}(\tilde{w}) \cup \mathsf{Comp}_{\text{conj}}(\tilde{w})$, set: $J^{\alpha} \leftarrow \perp$.

4.4.4.4 Stage I: Non-market transition $\mathcal{T}_{\text{nm}}$

The non-market transition updates all components of the world state except the market state $\mathcal{M}_t$. Formally, we define a kernel

$$\mathcal{T}_{\text{nm}} : \ (\mathcal{W} \times \mathcal{A}_{\text{nm}}) \times \Sigma_{\mathcal{W}} \to [0, 1]$$

which is implemented as the following pushforward kernel:

$$\mathcal{T}_{\text{nm}}(U \mid w, a_{\text{nm}}) \ = \ \int_{Y(\widetilde{w})} \mathbf{1}[\text{Post}_{\text{nm}}(\widetilde{w}, y) \in U] \ \mathbb{K}_{\widetilde{w}}(dy)$$

For measurable $U \subseteq \mathcal{W}$, $\widetilde{w} := \text{Pre}(w, a_{\text{nm}})$.

4.4.4.5 Stage II: Market transition via $\mathscr{M}$

The market mechanism $\mathscr{M}$ supplies the market transition kernel $\mathcal{T}_{\text{mkt}}$. Given the realized non-market update $w'_t \sim \mathcal{T}_{\text{nm}}(\cdot \mid w_t, a_{t,\text{nm}})$ and the market joint action $a_{t,\text{mkt}}$, we sample the final next state by

$$w_{t+1} \sim \mathcal{T}_{\text{mkt}}(\cdot \mid w_t, w'_t, a_{t,\text{mkt}}).$$

Intuitively, $\mathcal{T}_{\text{mkt}}$ may update portfolios $\{\mathcal{P}^{\alpha}\}$ and the public ledger $\mathcal{ML}$ as a function of both the submitted market actions and the realized mathematical progress encoded in the state change $w_t \to w'_t$, e.g. settlement on newly published proofs, bounty claims, liquidations, AMM price updates, etc. Of course, however, the actual implementation of $\mathcal{T}_{\text{mkt}}$ is entirely dependent on the specification of the market mechanism $\mathscr{M}$.

4.4.4.6 Full transition kernel

Finally, we may define, for any measurable $U \subseteq \mathcal{W}$,

$$\mathcal{T}(U \mid w_t, a_t) = \int_{\mathcal{W}} \mathcal{T}_{\text{mkt}}(U \mid w_t, w', a_{t,\text{mkt}}) \ \mathcal{T}_{\text{nm}}(dw' \mid w_t, a_{t,\text{nm}})$$

4.4.5 Reward, Discount Factor, and Initial Distribution

We take the per-step reward to be the (expected) *change in balance*:

$$r^\alpha(w, a) := \mathbb{E}[B^\alpha(w') - B^\alpha(w) \mid w' \sim \mathcal{T}(\cdot \mid w, a)]$$

Where $B^\alpha : \mathcal{W} \to \mathbb{R}$ is given by $B^\alpha(w) = \beta(P^\alpha(w))$, where $P^\alpha(w)$ is the usual projection of agent α's portfolio from the $\mathcal{M}$ component of $w \in \mathcal{W}$ and β is defined by the $\mathscr{M}$ component of $w \in \mathcal{W}$.

Finally, b_0 is a distribution over initial world states $w_0 \sim b_0$ specifying initial private/shared libraries, initial job statuses (typically idle), initial solve-time maps (typically empty), initial query models Q_0^α, and the initial market state, supported on states satisfying all invariants.

4.4.6 Consistency and well-definedness

We note that we require a VAMP to be *consistent* with its market mechanism $\mathscr{M}$, meaning that if we have a market mechanism:

$$\mathscr{M} = \Big(\mathcal{N}, \mathfrak{M}, \beta, \mathcal{W}, \{\mathcal{A}^\alpha_{\text{mkt}}\}_{\alpha \in \mathcal{N}}, \\ \{\text{Adm}^\alpha_{\text{mkt}}\}_{\alpha \in \mathcal{N}}, \mathcal{T}_{\text{mkt}}\Big)$$

The following tuple:

$$\Big(\mathcal{N}', \mathcal{W}', \{\mathcal{O}^\alpha\}_{\alpha \in \mathcal{N}}, \{\mathcal{Z}^\alpha\}_{\alpha \in \mathcal{N}}, \{\mathcal{A}^\alpha\}_{\alpha \in \mathcal{N}}, \\ \{\text{Adm}^\alpha\}_{\alpha \in \mathcal{N}}, \mathcal{T}, \{r^\alpha\}_{\alpha \in \mathcal{N}}, \gamma, b_0, \mathscr{M}\Big)$$

is a VAMP if and only if $\mathcal{N}' = \mathcal{N}, \mathcal{W}' = \mathcal{W}$, $\mathcal{A}^\alpha_{\text{mkt}} \subseteq \mathcal{A}^\alpha$, for $a \in \mathcal{A}^\alpha_{\text{mkt}}$ and $w \in \mathcal{W}$,$\text{Adm}^\alpha_{\text{mkt}}(w, a) = \text{Adm}^\alpha(w, a)$.

In other words, we may consider the following generator:

$$\Big(\mathcal{N}, \mathfrak{P}, \mathfrak{L}, \beta, \{\mathcal{A}^\alpha_{\text{mkt}}\}_{\alpha \in \mathcal{N}}, \{\text{Adm}^\alpha_{\text{mkt}}\}_{\alpha \in \mathcal{N}}, \mathcal{T}_{\text{mkt}}, \gamma, b_0\Big)$$

Which uniquely specifies a well-defined VAMP using the above formulation.

4.5 Decentralized ITAP-VAMP Correspondence

4.6 Equilibria and Tractibility

Chapter 5

Multiagent Reinforcement Learning for VAMPs

Chapter 6

Implementation

This section describes a concrete implementation of a restricted subset of VAMP's, with the goal of preserving the core dynamics while making simulation and learning tractable and realistic.

6.1 Assumptions

We implement a family of environments under the following simplifying assumptions.

6.1.0.1 Finite Formula Universe.

Each environment instance is defined over a finite universe of formulas (nodes) $\mathcal{F}$. All sets derived from $\mathcal{F}$ (ghost/concrete partitions, candidate actions, etc.) are therefore finite.

6.1.0.2 Finite Agent Universe.

We assume $\mathcal{N} = \{1, \cdots, n\}$ for some fixed $n \in \mathbb{N}$.

6.1.0.3 Truth and Difficulty Initialization.

We assume we may initialize the VAMP with a latent truth labeling over formulas $T : \mathcal{F} \to \{0, 1\}$, such that for all $\phi \in \mathcal{F}$, $T(\phi) + T(\neg\phi) = 1$.
We interpret $T(\phi) = 1$ as "ϕ is true/valid in the instance" and $T(\phi) = 0$ as "ϕ is false". For concision, we define $\mathcal{F}_\top := \{\phi \in \mathcal{F} : T(\phi) = 1\}$. Of course, if $[\Gamma \vdash \phi] \in \mathcal{S}$ is a resolved sequent then $\phi \in \mathcal{F}_\top$. This truth labeling is fixed over time and unobserved by agents except through interaction outcomes.
Additionally, we initialize a static latent difficulty map

$$d : \mathcal{F}_\top \to (0, 1)$$

used by the proof kernel as a (unobservable) hyperparameter in assigning success/failure probabilities heterogeneously. d is supported over $\{\phi \in \mathcal{F} : T(\phi) = 1\}$.

6.1.0.4 Static dependency structure and metadata.

We assume there exists a fixed, instance-dependent *ground truth* dependency mapping over formulas

$$\delta^\star : \mathcal{F}_\top \to \mathcal{P}_{\text{fin}}(\mathcal{F})$$

where we overload this notation to note for resolved sequents $[\Gamma \vdash \phi]$ via $\delta^\star([\Gamma \vdash \phi]) := \delta^\star(\phi) \subseteq \Gamma$ and such that the induced dependency digraph is acyclic. Then we may define for $s \in \mathcal{RS}^\alpha_{t_0}$ for all $t \geq t_0$, $\delta^\alpha_t(s) = \delta^\star(s)$.
We note this is simply an environment modeling choice. We do not claim mathematical proofs are unique; rather, it yields clean initialization and unambiguous progress signals.

6.1.0.5 Utility Graph.

Each instance of a VAMP initializes from a weighted directed graph

$$G_U := (\mathcal{F}_\top, E_U, w), \qquad w : E_U \to (0, 1],$$

where $w(\psi, \phi)$ measures the "utility" of ψ toward resolving or reasoning about ϕ. This is necessarily an informal notion that simply serves to allow us to highly related concepts that are not formally dependent on one another. We extend w by $w(\psi, \phi) = 0$ when $(\psi, \phi) \notin E_U$. We additionally note that:

$$w(\psi, \phi) = 1 \iff \psi \in \delta^*(\phi)$$

(So, if $E^\star = \{(\psi, \phi) \in E_U : w(\psi, \phi) = 1\} \subseteq E_U$, then the subgraph $G^\star = (\mathcal{F}_\top, E^\star)$ is a DAG).

6.1.0.6 Sequent Specification.

To prevent an exponential blowup of action space size due to agents' selection of sequents $[\Gamma \vdash \phi]$ for $\Gamma \subseteq_{\text{fin}} \mathcal{F}$, we automatically construct antecedent context based on the local library state. Namely, our implemented interface restricts agents to selecting only a consequent formula ϕ. The antecedent used to evaluate the attempt is then deterministically constructed from the agent's current local library state.
For agent α at time t, define the set of locally available resolved formulas

$$\mathcal{R}^\alpha_t := \left\{\psi \in \mathcal{F} : \exists \Gamma' \subseteq_{\text{fin}} \mathcal{F} \text{ such that } [\Gamma' \vdash \psi] \in \mathcal{RS}^\alpha_t\right\},$$

Now, we may redefine the (non-market) action space:

$$
\begin{aligned}
\widetilde{\mathcal{A}^{\alpha}_{\text{nm}}} :=& \{\text{Prove}\} \times (\mathcal{F} \times \mathbb{N}^{+}) \\
& \sqcup \{\text{Conj}\} \times (\mathcal{F} \times \mathbb{N}^{+}) \\
& \sqcup \{\text{Pub}\} \times \mathcal{F} \\
& \sqcup \{\text{Qry}\} \times \mathcal{S} \\
& \sqcup \{\perp_{\text{nm}}\}
\end{aligned}
$$

We note that the query action still relies on sequents rather than formulas, though we will shortly bound the admissible input sequents for this action in particular to allow for polynomial growth of the admissible action space size, while maintaining the ability of agents to "suppose lemma ψ is proven" when running a query. W fix a hyperparameter $h_{\max}$ to denote this boundedness of the admissible antecedents' extension beyond $\mathcal{R}^{\alpha}_{t}$.
Additionally, as the market mechanism action space is abstractly defined, we note that it is up to the user specification to decide whether similar implementation changes must be changed in $\mathcal{A}_{\text{mkt}}$ as well.
Additionally, we update the specification of the admissibility criteria, where for $w_t \in \mathcal{W}$ and $\alpha \in \mathcal{N}$, if $J^{\alpha}_{t} = \perp$, then:

- For $a^{\alpha}_{t} = (\text{Prove}, (\phi, \tau))$ and $a^{\alpha}_{t} = (\text{Conj}, (\phi, \tau))$:

$$\widetilde{\text{Adm}^{\alpha}}(w_t, a^{\alpha}_{t}) = 1 \iff \phi \in \mathcal{C}^{\alpha}_{t}$$

- For $a^{\alpha}_{t} = (\text{Pub}, \phi)$:

$$\widetilde{\text{Adm}^{\alpha}}(w_t, a^{\alpha}_{t}) = 1 \iff \phi \in \mathcal{C}^{\alpha}_{t}$$

- For $a^{\alpha}_{t} = (\text{Qry}, [\Gamma \vdash \phi])$:

$$
\begin{aligned}
\widetilde{\text{Adm}^{\alpha}}(w_t, a^{\alpha}_{t}) = 1 \iff & [\Gamma \vdash \phi] \in \mathcal{CS}^{\alpha}_{t} \\
& \wedge (|\Gamma \setminus \mathcal{R}^{\alpha}_{t}| \leq h_{\max})
\end{aligned}
$$

Where in all other cases, $\widetilde{\text{Adm}^{\alpha}}(w_t, a^{\alpha}_{t}) = \text{Adm}^{\alpha}(w_t, a^{\alpha}_{t})$.
Then, given $w_t \in \mathcal{W}$ and admissible $\widetilde{a^{\alpha}_{t}} \in \widetilde{\mathcal{A}^{\alpha}_{t}}(w_t)$, we may recover a corresponding $a^{\alpha}_{t} \in \mathcal{A}^{\alpha}_{t}(w_t)$, defined as follows:

- If $\widetilde{a^{\alpha}_{t}} = (\text{Prove}, \phi, \tau)$ then $a^{\alpha}_{t} = (\text{Prove}, [\mathcal{R}^{\alpha}_{t} \vdash \phi], \tau)$.

- If $\widetilde{a^{\alpha}_{t}} = (\text{Conj}, \phi, \tau)$ then $a^{\alpha}_{t} = (\text{Conj}, [\mathcal{R}^{\alpha}_{t} \vdash \phi], \tau)$.

- If $\widetilde{a_t^\alpha} = (\text{Pub}, \phi)$ then $a_t^\alpha = (\text{Pub}, \{\phi\}, \{[\mathcal{R}_t^\alpha \vdash \phi]\} \cap \mathcal{R}_t^\alpha)$.
- Otherwise, $\widetilde{a_t^\alpha} = a_t^\alpha$.

We note that this notion of fixing a canonical local context $\mathcal{R}_t^\alpha$ effectively defines a bijection from (concrete) formulas to (concrete) sequents, in that if we define

$$s_t^\alpha(\phi) \;:=\; [\Gamma_t^\alpha \vdash \phi] \;\in\; \mathcal{S}.$$

$$\widetilde{\mathcal{CS}}_t^\alpha := \{\, s_t^\alpha(\phi) : \phi \in \mathcal{C}_t^\alpha \,\},$$

and the map $\phi \mapsto s_t^\alpha(\phi)$ is a bijection between $\mathcal{C}_t^\alpha$ and $\widetilde{\mathcal{CS}}_t^\alpha$.
Additionally, we note that this assumption automatically implies that all resolved sequents are actually fully resolved.

6.2 Proof Kernel ($\mathcal{K}_{\text{proof}}$).

The proof kernel models the outcome of spending compute budget on an admissible proof attempt. For an agent α and its library state $\mathcal{L} \in \mathscr{L}$, target sequent $s \in \mathcal{S}$ (though in practice $s \in \mathcal{CS}^\alpha$), and compute budget $\tau \in \mathbb{N}^+$, the kernel outputs a binary success indicator and set of dependencies. However, as we assume a global dependency set, we may simply consider $\mathcal{K}_{\text{proof}}$ outputting in $\{0, 1\}$, with the dependency component being computed deterministically as $\delta^\star(s)$.
With that in mind, we now proceed to explicitly define our particular implementation of $\mathcal{K}_{\text{proof}}$, stepping through its parameterization and desired behaviors. Namely, we construct $\mathcal{K}_{\text{proof}}$ to have the following properties:

6.2.0.1 Bernoulli proof outcomes.

We assume proof attempts have only binary stochasticity. Concretely, $\mathcal{K}_{\text{proof}}^\alpha(\cdot \mid \mathcal{L}, s, \tau)$ is supported on exactly two atoms:

$$\mathcal{K}_{\text{proof}}^\alpha\big((0, \bot) \mid \mathcal{L}, s, \tau\big) = 1 - p_{\text{proof}}^\alpha(s, \tau)$$

$$\mathcal{K}_{\text{proof}}^\alpha\big((1, \delta^\star(s)) \mid \mathcal{L}, s, \tau\big) = p_{\text{proof}}^\alpha(s, \tau)$$

and assigns zero mass to all $(1, d)$ with $d \neq \delta^\star(s)$.

6.2.0.2 Truth + Dependency Gate.

If ϕ is false (i.e. $T(\phi) = 0$), the probability of success is 0. Moreover, if $\delta^\star(\phi) \not\subseteq \mathcal{R}_t^\alpha$, success probability is 0. In other words, one cannot prove a theorem before its hard prerequisites.

Towards this, we may define a feasibility gate

$$g([\Gamma \vdash \phi]) = \mathbf{1}[T(\phi) = 1] \cdot \mathbf{1}[\delta^\star(\phi) \subseteq \Gamma],$$

6.2.0.3 Budget Monotonicity and Diminishing Returns.

We enforce that $p^\alpha_{\text{proof}}(s, \cdot)$ is nondecreasing, intuitively modeling that more cumulative compute can't hurt success likelihood. Additionally, we hope to model diminishing returns with increasing budget such that marginal gains decrease with more budget. This serves to prevent undesirable strategies such as "always dump infinite compute into one node". A natural implementation of this is to model:

$$p^\alpha_{\text{proof}}([\Gamma \vdash \phi], \tau) = K_0 \left(1 - e^{-K_1 \cdot h(\tau)}\right)$$

For some $K_0, K_1 \geq 0$, and a concave h. In practice, one often chooses $h(\tau) = \tau^\kappa$ governed by a hyperparameter $\kappa \in (0, 1]$.

6.2.0.4 Difficulty Modulation.

We recall that we initialize $\mathcal{F}_\top$ with a latent difficulty map $d : \mathcal{F}_\top \to (0, 1)$. Given the same time budget τ, harder theorems are less likely to be solved. Moreover, difficulty should scale the rate (hazard) rather than just shifting probabilities ad hoc. Towards this notion, define:

$$r_{\text{diff}}(\phi) = e^{-\lambda_{\text{diff}}\, d(\phi)}$$

Governed by a difficulty-to-rate transform $\lambda_{\text{diff}} > 0$.

6.2.0.5 Utility Modulation.

We also recall that we initialize a utility graph G_U over $\mathcal{F}_\top$ as a weighted digraph. Although the feasibility gate g filters sequents $[\Gamma \vdash \phi]$ based on the inclusion of the hard dependencies $\delta^\star$, we aim to model the notion that even once hard dependencies are met, having additional relevant resolved lemmas in Γ should increase success probabilities. This too should be monotone and saturating. Towards this, we define:

$$s_{\text{mass}}([\Gamma \vdash \phi]) = \sum_{\psi \in \Gamma} w(\psi, \phi)^\alpha$$

with $\alpha \geq 1$, and a saturating transform

$$s([\Gamma \vdash \phi]) = \log\left(1 + s_{\text{mass}}([\Gamma \vdash \phi])\right)$$

This design preserves the distinction between having one useful theorem and many useful theorems, while preventing unbalanced growth in success rate. It is governed by a hyperpa-

rameter $\alpha \geq 1$ to affects the sensitivity to high-utility neighbors.

6.2.0.6 Heterogeneous Agent Capability Modulation.

We initialize agents with capability parameters:

- $\rho_0^\alpha \geq 0$ representing agent α's baseline solve-rate
- $\rho_1^\alpha \geq 0$ representing agent α's amplification from high-utility support

With this in mind, a natural model of the effective proof rate of agent α on a particular target sequent is:

$$\rho^\alpha([\Gamma \vdash \phi]) = r_{\text{diff}}(\phi)\Big(\rho_0^\alpha + \rho_1^\alpha s([\Gamma \vdash \phi])\Big)$$

6.2.1 Definition

As such, we may explicitly implement:

$$\mathcal{K}^\alpha_{\text{proof}}\big((0, \bot) \mid \mathcal{L}, s, \tau\big) = 1 - p^\alpha_{\text{proof}}(s, \tau)$$

$$\mathcal{K}^\alpha_{\text{proof}}\big((1, \delta^\star(s)) \mid \mathcal{L}, s, \tau\big) = p^\alpha_{\text{proof}}(s, \tau)$$

and assigns zero mass to all $(1, d)$ with $d \neq \delta^\star(s)$, where:

$$p^\alpha_{\text{proof}}([\Gamma \vdash \phi], \tau) = g([\Gamma \vdash \phi])\left(1 - e^{-\rho^\alpha([\Gamma \vdash \phi]) \cdot h(\tau)}\right)$$

We note that $p^\alpha_{\text{proof}}(\cdot) \in [0, 1]$ for all inputs, and success probability is strictly increasing and approaches 1 as $\tau_{\text{eff}} \to \infty$ on feasible inputs that pass g.

6.3 Conjecture Kernel ($\mathcal{K}_{\text{conj}}$).

The conjecture kernel models the outcome of allocating compute budget toward proposing a new formula. Given an agent $\alpha \in \mathcal{N}$, a library state $\mathcal{L} = (\mathcal{C}, \mathcal{RS}, \delta, \Theta) \in \mathscr{L}$, an anchor sequent $s = [\Gamma \vdash \phi] \in \mathcal{S}$ (though in practice $s \in \mathcal{CS}^\alpha$), and budget $\tau \in \mathbb{N}^+$, the conjecture kernel outputs either failure or success together with a proposed formula $\psi \in \mathcal{F}$. However, successful outputs are required to land in the current ghost set $\mathcal{G}(\mathcal{L}) := \mathcal{F} \setminus \mathcal{C}$.
We now define our particular implementation of $\mathcal{K}^\alpha_{\text{conj}}$, stepping through its desired behaviors and parameterization.

6.3.0.1 Bernoulli success with conditional proposal.

We assume conjecturing has binary stochasticity at the success/failure level, followed by a proposal distribution on success. Concretely, there exists a success probability

$$p^{\alpha}_{\text{conj}}(\mathcal{L}, s, \tau) \in [0, 1]$$

and a conditional proposal distribution

$$\pi^{\alpha}_{\text{conj}}(\cdot \mid \mathcal{L}, s, \tau)$$

supported on $\mathcal{G}(\mathcal{L})$ such that

$$\mathcal{K}^{\alpha}_{\text{conj}}\big((0, \perp) \mid \mathcal{L}, s, \tau\big) = 1 - p^{\alpha}_{\text{conj}}(\mathcal{L}, s, \tau),$$

and for $\psi \in \mathcal{F}$,

$$\mathcal{K}^{\alpha}_{\text{conj}}\big((1, \psi) \mid \mathcal{L}, s, \tau\big) = p^{\alpha}_{\text{conj}}(\mathcal{L}, s, \tau)\, \pi^{\alpha}_{\text{conj}}(\psi \mid \mathcal{L}, s, \tau).$$

6.3.0.2 Budget monotonicity and diminishing returns.

As with the proof kernel, we require $p^{\alpha}_{\text{conj}}(\mathcal{L}, s, \cdot)$ to be nondecreasing with diminishing returns. We therefore model

$$p^{\alpha}_{\text{conj}}(\mathcal{L}, s, \tau) = 1 - \exp\big(- \eta^{\alpha}(\mathcal{L}, s)\, h(\tau)\big),$$

where $h : \mathbb{N}^{+} \to \mathbb{R}_{\geq 0}$ is increasing and concave. In practice, we again use

$$h(\tau) = \tau^{\kappa}, \qquad \kappa \in (0, 1].$$

6.3.0.3 Heterogeneous agent conjecturing capability.

We equip each agent α with conjecturing capability parameters

$$\eta^{\alpha}_{0}, \eta^{\alpha}_{1} \geq 0,$$

representing baseline conjecture rate and amplification from local graph opportunity, respectively.

6.3.0.4 Utility-guided relevance.

We recall the instance utility graph

$$G_U = (\mathcal{F}_{\top}, E_U, w),$$

where $w(\psi, \phi)$ measures the utility of ψ toward resolving or reasoning about ϕ, extended by 0 off E_U.

Intuitively, useful conjectures should depend on the role of the anchor consequent ϕ in the current local library. If ϕ is already resolved, then good conjectures should tend to extend outward from ϕ; if ϕ is not yet resolved, then good conjectures should tend to provide lemmas that support ϕ.

and define the anchor-conditioned utility score

$$q(\psi \mid \mathcal{L}, [\Gamma \vdash \phi]) = \mathbf{1}[[\Gamma \vdash \phi] \in \mathcal{RS}^\alpha]\, w(\phi, \psi) + \mathbf{1}[[\Gamma \vdash \phi] \notin \mathcal{RS}^\alpha]\, w(\psi, \phi).$$

Thus, when ϕ is resolved, the score favors frontier extensions; when ϕ is unresolved, it favors potentially useful supporting lemmas.

6.3.0.5 Local opportunity modulation.

We also want conjecturing to be easier in regions where there exist many promising unseen formulas. To capture this, define the local opportunity mass

$$m^\alpha(\mathcal{L}, s) := \sum_{\psi \in \mathcal{G}(\mathcal{L})} \big(q(\psi \mid \mathcal{L}, s)\big)^{\beta_{\text{conj}}}, \qquad \beta_{\text{conj}} \geq 1,$$

and the saturating transform

$$u^\alpha(\mathcal{L}, s) := \log\big(1 + m^\alpha(\mathcal{L}, s)\big).$$

We then define the effective conjecturing rate by

$$\eta^\alpha(\mathcal{L}, s) = \eta_0^\alpha + \eta_1^\alpha\, u^\alpha(\mathcal{L}, s).$$

6.3.0.6 Budget-improved proposal quality.

Beyond increasing the chance of any successful conjecture, additional compute should also improve the quality of the proposed conjecture conditional on success. We model this by allowing the proposal distribution to become more concentrated around high-utility candidates as budget increases.

To do so, define a budget-dependent selectivity parameter

$$\kappa_{\text{conj}}^\alpha(\tau) = \kappa_0^\alpha + \kappa_1^\alpha h(\tau), \qquad \kappa_0^\alpha, \kappa_1^\alpha \geq 0.$$

Small values of $\kappa_{\text{conj}}^\alpha(\tau)$ yield diffuse proposal distributions, while large values concentrate mass on high-score candidates.

6.3.0.7 Proposal distribution over all formulas, restricted to ghosts by renormalization.

It is convenient to first define a raw proposal score over all formulas $\psi \in \mathcal{F}$, then restrict to the current ghost set induced by $\mathcal{L}$.
Let

$$r^{\alpha}(\psi \mid \mathcal{L}, s, \tau) := \exp\Big(\kappa^{\alpha}_{\text{conj}}(\tau)\, \varphi(q(\psi \mid \mathcal{L}, s))\Big),$$

where $\varphi : [0,1] \to \mathbb{R}_{\geq 0}$ is a fixed monotone transform (for instance $\varphi(x) = x$ or $\varphi(x) = \log(1+x)$).
We then define the actual proposal distribution by renormalizing over the current ghost set:

$$\pi^{\alpha}_{\text{conj}}(\psi \mid \mathcal{L}, s, \tau) = \frac{r^{\alpha}(\psi \mid \mathcal{L}, s, \tau)\, \mathbf{1}[\psi \in \mathcal{G}(\mathcal{L})]}{\sum_{\psi' \in \mathcal{G}(\mathcal{L})} r^{\alpha}(\psi' \mid \mathcal{L}, s, \tau)}.$$

Hence the conjecturer is specified globally over $\mathcal{F}$, but at runtime only samples from formulas that are currently ghost relative to $\mathcal{L}$.

6.3.1 Definition

Putting the above together, for an admissible conjecture attempt on $s = [\Gamma \vdash \phi]$ with budget τ, we define

$$\mathcal{K}^{\alpha}_{\text{conj}}\big((0, \bot) \mid \mathcal{L}, s, \tau\big) = 1 - p^{\alpha}_{\text{conj}}(\mathcal{L}, s, \tau),$$

and for $\psi \in \mathcal{F}$,

$$\mathcal{K}^{\alpha}_{\text{conj}}\big((1, \psi) \mid \mathcal{L}, s, \tau\big) = p^{\alpha}_{\text{conj}}(\mathcal{L}, s, \tau)\, \pi^{\alpha}_{\text{conj}}(\psi \mid \mathcal{L}, s, \tau).$$

By construction, $\pi^{\alpha}_{\text{conj}}(\cdot \mid \mathcal{L}, s, \tau)$ is supported on $\mathcal{G}(\mathcal{L})$, $p^{\alpha}_{\text{conj}}(\mathcal{L}, s, \cdot)$ is monotone with diminishing returns, and larger budgets increase both the chance of successful conjecturing and the concentration of mass on high-utility conjectures.

6.4 Query Model.

The query model is a stateful predictive module used by each agent to estimate how its prover process is expected to perform on a target sequent under its current local library state. In our implementation, the query model does *not* estimate semantic truth directly. Rather, for agent α, it estimates:

1. the probability that α will *eventually resolve* a target sequent under its prover process, and

2. the expected *additional compute budget* required to do so, conditional on eventual success.

Accordingly, we instantiate the query readout as a map

$$q^\alpha : \mathcal{Q}^\alpha \times \mathscr{L} \times \mathcal{S} \to [0,1] \times \mathbb{N},$$

where

$$q^\alpha(Q_t^\alpha, \mathcal{L}, s) = \big(\hat{p}_t^\alpha(\mathcal{L}, s), \hat{\tau}_t^\alpha(\mathcal{L}, s)\big).$$

Here:

- $\hat{p}_t^\alpha(\mathcal{L}, s)$ estimates the probability that agent α's prover process will eventually resolve s from local library state $\mathcal{L}$,
- $\hat{\tau}_t^\alpha(\mathcal{L}, s)$ estimates the expected additional budget required to resolve s, conditional on eventual success.

6.4.0.1 Feature buckets.

To make online updates statistically stable while preserving algebraic simplicity, we use a finite bucketization of query instances. For each agent α, fix a finite bucket set $\mathcal{B}^\alpha$ and a deterministic bucket map

$$b^\alpha : \mathscr{L} \times \mathcal{S} \to \mathcal{B}^\alpha.$$

The bucket map may depend on any collection of state/query features, such as:

- support mass or utility features of the target,
- dependency-completion indicators,
- target difficulty proxies,
- cumulative prior budget spent on the target,
- local graph-topological features,
- other agent-specific or state-specific summary statistics.

The framework does not prescribe the exact feature design.

6.4.0.2 Discrete-time survival model.

Fix a finite horizon $H \in \mathbb{N}^+$. For each agent α and bucket $b \in \mathcal{B}^\alpha$, we model a discrete-time hazard sequence

$$\lambda_b^\alpha(k) \in [0,1], \qquad k = 1, \ldots, H,$$

where

$$\lambda_b^\alpha(k) := \Pr(T_s^\alpha = k \mid T_s^\alpha \geq k,\ b^\alpha(\mathcal{L}, s) = b).$$

Here T_s^α denotes the random additional budget required for agent α to resolve s. Equivalently, conditional on bucket b, the discrete survival function is

$$S_b^\alpha(k) := \Pr(T_s^\alpha > k \mid b) = \prod_{j=1}^{k} \big(1 - \lambda_b^\alpha(j)\big), \qquad S_b^\alpha(0) := 1.$$

Thus the probability of solving exactly at budget step k is

$$\Pr(T_s^\alpha = k \mid b) = \lambda_b^\alpha(k) \prod_{j=1}^{k-1} (1 - \lambda_b^\alpha(j)).$$

6.4.0.3 Query readout.

Given current state $(Q_t^\alpha, \mathcal{L}, s)$, let

$$b = b^\alpha(\mathcal{L}, s).$$

Then the query model returns:

$$\hat{p}_t^\alpha(\mathcal{L}, s) = \Pr(T_s^\alpha \leq H \mid b) = 1 - S_b^\alpha(H) = 1 - \prod_{j=1}^{H} (1 - \lambda_b^\alpha(j)),$$

which is the estimated probability of eventual solve success within the modeling horizon H. For the conditional expected additional budget, define

$$\hat{\tau}_t^\alpha(\mathcal{L}, s) = \mathbb{E}[T_s^\alpha \mid T_s^\alpha \leq H,\ b] = \frac{\sum_{k=1}^{H} k \ \Pr(T_s^\alpha = k \mid b)}{\hat{p}_t^\alpha(\mathcal{L}, s)},$$

whenever $\hat{p}_t^\alpha(\mathcal{L}, s) > 0$. If $\hat{p}_t^\alpha(\mathcal{L}, s) = 0$, we may return a fixed sentinel value (for instance H or $+\infty$ in an abstract implementation).

6.4.0.4 Query model state.

For each agent α, the query model state

$$Q_t^\alpha \in \mathcal{Q}^\alpha$$

stores, for every bucket $b \in \mathcal{B}^\alpha$ and time index $k \in \{1, \ldots, H\}$, two nonnegative sufficient statistics:

$$N_t^\alpha(b,k), \qquad D_t^\alpha(b,k).$$

These are interpreted as:

- $D_t^\alpha(b,k)$ = the number of observed proof-attempt trajectories in bucket b that remained "at risk" at budget step k,
- $N_t^\alpha(b,k)$ = the number of those trajectories that terminated successfully exactly at budget step k.

6.4.0.5 MAP-smoothed hazard estimates.

To ensure numerical stability and prevent degenerate zero/one hazards early in training, we introduce fixed hyperparameters

$$a_{b,k}^\alpha > 0, \qquad c_{b,k}^\alpha > 0$$

for each bucket-time pair. These act as pseudocounts. We then define the hazard estimate by the smoothed ratio

$$\lambda_b^\alpha(k) = \frac{N_t^\alpha(b,k) + a_{b,k}^\alpha}{D_t^\alpha(b,k) + a_{b,k}^\alpha + c_{b,k}^\alpha}.$$

This may be interpreted as the coordinatewise MAP estimate of a Bernoulli hazard with Beta prior.

6.4.0.6 Observed data and censoring.

Each proof attempt produces an observation datum for the querying agent. We model a datum as

$$d = (b, \tau, \xi) \in \mathcal{B}^\alpha \times \mathbb{N}^+ \times \{0,1\},$$

where:

- b is the feature bucket of the queried instance at attempt time,
- τ is the additional budget consumed in that attempt,
- $\xi = 1$ indicates that the attempt succeeded exactly at budget τ,

- $\xi = 0$ indicates that the attempt was censored (timed out / did not solve) at budget τ.

Such a datum contributes additive increments to the sufficient statistics:

$$D_t^\alpha(b,k) \leftarrow D_t^\alpha(b,k) + 1 \qquad \text{for all } 1 \le k \le \min\{\tau, H\},$$

and, if $\xi = 1$ and $\tau \le H$, additionally

$$N_t^\alpha(b,\tau) \leftarrow N_t^\alpha(b,\tau) + 1.$$

Hence unsuccessful attempts contribute exposure only, while successful attempts contribute both exposure and one event at the realized solve time.

6.4.0.7 Update rule.

Define the update operator

$$\Delta^\alpha : \mathcal{Q}^\alpha \times \mathcal{D}(\mathcal{Q}^\alpha) \to \mathcal{Q}^\alpha$$

to be the additive update of these sufficient statistics induced by a single datum d as above. Because the update is coordinatewise addition on count tables, it is exactly associative and commutative. Therefore, if multiple observations are received in a single timestep, the resulting batch update is well-defined independently of order.

6.5 Market Mechanisms

We now specify the family of concrete market mechanisms tested in this work. Each mechanism is implemented as an instantiation of the abstract market mechanism interface

$$\mathscr{M} = \Big(\mathcal{N}, \mathfrak{M}, \beta, \mathcal{W}, \{\mathcal{A}_{\text{mkt}}^\alpha\}_{\alpha\in\mathcal{N}}, \{\text{Adm}_{\text{mkt}}^\alpha\}_{\alpha\in\mathcal{N}}, \mathcal{T}_{\text{mkt}}\Big),$$

but we restrict attention to a common implementation template so that different mechanisms may be compared under the same underlying theorem-proving environment.

6.5.0.1 Common portfolio structure.

For all tested mechanisms, each agent α maintains a portfolio

$$\mathcal{P}_t^\alpha = (c_t^\alpha, H_t^\alpha, O_t^\alpha),$$

where:

- $c_t^\alpha \in \mathbb{R}_{\ge 0}$ is current cash,

- H_t^α is the multiset of currently held assets / claims,
- O_t^α is the multiset of currently posted market offers.

The public market ledger $\mathcal{ML}_t$ stores the union of all currently active public offers together with any mechanism-specific public state (for example AMM inventory variables or posted bounty books).

6.5.0.2 Worst-case collateralization.

To prevent agents from taking positions they cannot honor, we associate to each portfolio a worst-case marked balance

$$\underline{B}_t^\alpha := \underline{\beta}(\mathcal{P}_t^\alpha) \in \mathbb{R},$$

defined as the cash position minus the largest possible future liability of the held and posted claims under the mechanism's settlement rule. We require all admissible market actions to preserve

$$\underline{B}_t^\alpha \geq 0.$$

Thus agents are fully collateralized against worst-case future settlement.

6.5.0.3 Cash conservation.

All tested mechanisms satisfy internal money conservation at trade time: transferring an asset from one agent to another is accompanied by an equal and opposite transfer of cash, except where the mechanism explicitly introduces escrowed collateral or deterministic settlement transfers already implied by outstanding liabilities.

6.5.0.4 Public settlement events.

All tested mechanisms settle only against publicly observable events, namely changes in the public library $\mathcal{L}_t^0$. In particular, contract payouts may depend only on whether a target formula or sequent has been publicly resolved by a specified time.

6.5.0.5 Resolution events.

For a public target sequent s and deadline $T \in \mathbb{N}$, define the binary event

$$E_{s,T}(w_{0:\infty}) := \mathbf{1}\big[\exists\, t \leq T \text{ such that } s \in \mathcal{RS}_t^0\big].$$

Equivalently, $E_{s,T} = 1$ iff s is publicly resolved by time T. When working in the implemented formula-based interface, we identify a formula ϕ with its canonical public sequent

$$s_t^0(\phi) := [\mathcal{R}_t^0 \vdash \phi]$$

and write $E_{\phi,T}$ by abuse of notation.

6.5.1 Mechanism: Collateralized Bilateral Contract Market

As an initial work in the field of MARL on VAMP's, we focus on only one particular market mechanism and study its dynamics. We do not claim that such a market mechanism is globally optimal, but rather focus on it as a simple and robust communication substrate in such verified domains such as formal mathematics.

Namely, our market mechanism is a posted-offer market over binary contingent contracts. A *contract type* is a triple

$$\chi = (s, T, \ell) \in \mathcal{RS}^{0,\text{cand}} \times \mathbb{N} \times [0, 1],$$

where:

- s is a public target sequent,
- T is a strike / deadline time,
- $\ell \in [0, 1]$ is the maximum loss parameter.

Intuitively, $\chi = (s, T, \ell)$ is a normalized unit-notional claim on the event that s becomes publicly resolved by time T.

6.5.1.1 Long and short claims.

Each contract type χ induces two complementary positions:

$$\chi^+ \qquad \text{and} \qquad \chi^-,$$

called the long and short side, respectively. Their terminal cash payoffs are:

$$\Pi(\chi^+) = \begin{cases} 1 - \ell, & E_{s,T} = 1, \\ -\ell, & E_{s,T} = 0, \end{cases} \qquad \Pi(\chi^-) = -\Pi(\chi^+).$$

Thus the long side gains when the target resolves by the deadline, while the short side gains otherwise. The magnitude of the downside of the long position is exactly ℓ, and the upside is $1 - \ell$.

6.5.1.2 Unit notional and quantities.

Contracts are traded in nonnegative integer quantities. A position of quantity $q \in \mathbb{N}$ in χ^+ has payoff $q\,\Pi(\chi^+)$, and similarly for χ^-. Hence larger notional exposure is represented by larger quantities rather than by changing the contract definition itself.

6.5.1.3 Asset creation.

A new quantity-q contract issue of type χ is created only as a matched pair:

$$q\chi^+ \quad \text{and} \quad q\chi^-.$$

This pair may initially be placed into the issuing agent's holdings or posted offers. Since the long and short payoffs sum to zero statewise, pair creation introduces no net wealth.

6.5.1.4 Public offers.

An offer is a tuple

$$o = (\chi, \sigma, q, p),$$

where χ is a contract type, $\sigma \in \{+, -\}$ is the side being offered, $q \in \mathbb{N}^+$ is quantity, and $p \in [0, 1]$ is the per-unit transaction price. Posted offers are stored in the public ledger $\mathcal{ML}_t$.

6.5.1.5 Trades.

When agent β accepts quantity $q' \leq q$ from an offer $o = (\chi, \sigma, q, p)$ posted by agent α, the ledger removes $q'\chi^\sigma$ from α's posted offers and transfers it to β's holdings, while cash is transferred in the opposite direction:

$$c^\beta \leftarrow c^\beta - q'p, \qquad c^\alpha \leftarrow c^\alpha + q'p.$$

The residual quantity $q - q'$ remains posted if positive.

6.5.1.6 Collateral and admissibility.

For a held or posted position x, define its worst-case liability by

$$L(x) := \sup_\omega (-\Pi_\omega(x))_+.$$

For a portfolio $\mathcal{P}^\alpha = (c^\alpha, H^\alpha, O^\alpha)$, define the minimum marked balance

$$\underline{B}^\alpha := c^\alpha - \sum_{x \in H^\alpha \cup O^\alpha} L(x).$$

A market action is admissible only if the resulting portfolio satisfies

$$\underline{B}^{\alpha} \geq 0.$$

Hence every agent is fully collateralized against the worst-case settlement of all currently held and posted obligations.

6.5.1.7 Settlement.

At each timestep, any contract type $\chi = (s, T, \ell)$ with deadline $T = t$ is settled against the public event $E_{s,T}$. For each agent, the corresponding cash payoff is applied to all held and posted quantities of $\chi^{\pm}$, and the settled contracts are removed from portfolios and the public ledger.

TODO IMPLEMENTATION DETAILS FOR MARL TRAINING!!!!!!

Chapter 7

Experiments

TODO PASTE FROM OVERLEAF WRITEUP

Chapter 8

Application to Neural Theorem Proving

TODO PASTE FROM OVERLEAF WRITEUP

Chapter 9

Conclusion

Bibliography